
Федеральное государственное автономное образовательное учреждение
высшего образования

«Московский физико-технический институт
(национальный исследовательский университет)»
Физтех-школа Прикладной Математики и Информатики
Кафедра алгоритмов и технологий программирования

Направление подготовки / специальность: 09.04.01 Информатика и вычислительная техника

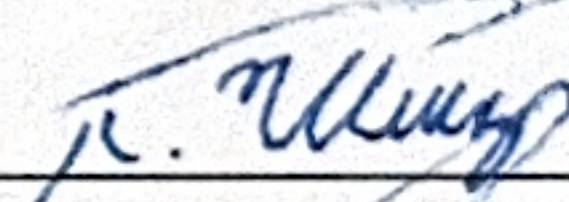
Направленность (профиль) подготовки: Прикладная математика и информатика

**РЕАЛИЗАЦИЯ ОТКАЗОУСТОЙЧИВОГО
МНОГОПОЛЬЗОВАТЕЛЬСКОГО L3/L4 БАЛАНСИРОВЩИКА
НАГРУЗКИ НА БАЗЕ VPP**

(магистерская диссертация)

Студент:

Шлыков Павел Сергеевич



(подпись студента)

Научный руководитель:

Цитович Иван Иванович,
д-р физ.-мат. наук, доц.

(подпись научного руководителя)

Консультант (при наличии):

(подпись консультанта)

Москва 2026

РЕФЕРАТ

Отчёт содержит 78 с., 8 рис., 29 источн.

Ключевые слова: БАЛАНСИРОВКА НАГРУЗКИ, VPP, ОБЛАКО, ОТКАЗОУСТОЙЧИВОСТЬ, CONTROL PLANE, DATA PLANE, HEALTH CHECK, CONSISTENT HASHING, KUBERNETES, CHAOS TESTING.

Объектом исследования являются облачные системы балансировки сетевого трафика на уровнях L3/L4.

Предметом исследования является архитектура, алгоритмы и методы обеспечения отказоустойчивости L3/L4 балансировщика нагрузки в облачной среде.

Цель работы — реализация отказоустойчивого многопользовательского L3/L4 балансировщика нагрузки на базе VPP, адаптированного под нужды облачного провайдера.

В настоящей работе была спроектирована, реализована и протестирована отказоустойчивая система сетевой балансировки нагрузки на базе VPP, в которую входят следующие компоненты: плоскость управления, плоскость данных и подсистема мониторинга состояния клиентских эндпоинтов.

К основным результатам работы относятся:

- анализ существующих решений в области L3/L4 балансировки, выявлено отсутствие полноценных открытых аналогов;
- компонентная архитектура системы с разделением на плоскость управления, плоскость данных и систему проверок состояния;
- плоскость управления с декларативным API, реконсиляцией размещения целевых групп, gRPC API с long-poll и версионированием через etcd;
- децентрализованная система проверок состояния на основе gossip/SWIM и двухуровневого consistent hashing с настраиваемым фактором репликации;

- агент плоскости данных с реконсильцией целевого состояния и подпиской на health-статусы через CDC;
- методика хаос-тестирования с воспроизводимыми сценариями отказов;
- серия экспериментов, подтвердивших отказоустойчивость разработанной системы.

Область применения - внутренняя инфраструктура облачного провайдера для предоставления сервиса L3/L4 балансировки нагрузки облачным управляемым решениям и конечным пользователям.

СОДЕРЖАНИЕ

РЕФЕРАТ	1
СОДЕРЖАНИЕ	3
ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	6
ВВЕДЕНИЕ	11
1 Обзор существующих решений	16
1.1 Google Maglev	16
1.2 Cloudflare Unimog	18
1.3 Yandex Cloud L3/L4 Load Balancer	19
1.4 Требования к целевому решению	21
1.5 Выводы	24
2 Архитектура решения	27
2.1 Компонентная архитектура	27
2.2 Модель данных	29
2.3 Паттерн реконсиляции	30
2.4 Потоки данных	32
3 Решения для плоскости данных	34
3.1 IPVS	34
3.2 Katran	35
3.3 VPP	35
3.4 Сравнение и выбор решения	37
4 Агент плоскости данных	38
4.1 Архитектура агента	38

4.2 Абстракция плоскости данных	39
4.3 Локальное хранилище.....	39
4.4 Реконсиляция спецификации	40
4.5 Реконсиляция эндпоинтов	41
4.6 Получение конфигурации от плоскости управления.....	41
4.7 Получение статусов здоровья	42
5 Плоскость управления	44
5.1 Обзор подходов к плоскости управления	44
5.2 Компоненты плоскости управления.....	45
5.3 Поток конфигурационных данных	47
5.4 Хранилище состояния.....	47
5.5 Модель версионирования	50
5.6 Реконсилитор	51
5.7 Алгоритм размещения целевых групп	52
5.8 Доставка конфигурации агентам	54
6 Система проверок состояния.....	55
6.1 Существующие подходы и их ограничения	55
6.2 Архитектура системы проверок.....	56
6.3 Распределение проверок между узлами.....	58
6.4 Реакция на события членства	59
6.5 Хранение и доставка результатов	60
6.6 Управление списком проверяемых целей.....	62
7 Тестирование	63

7.1 Функциональное тестирование	63
7.2 Нагрузочное тестирование	63
7.3 Хаос-тестирование, методология.....	66
7.4 Тестовый стенд	69
7.5 Метрики.....	70
7.6 Результаты.....	71
ЗАКЛЮЧЕНИЕ	76
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	79

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей работе применяют следующие термины с соответствующими определениями, обозначениями и сокращениями.

- 5-tuple hash — хеш-значение, вычисляемое от пяти полей сетевого соединения (протокол транспортного уровня, IP-адрес и порт источника, IP-адрес и порт назначения) и используемое для устойчивой привязки соединения к узлу обработки трафика.
- CDC (Change Data Capture) — паттерн захвата изменений данных в базе данных с последующей передачей этих изменений в потоковую систему обработки.
- Chaos testing — методика проверки отказоустойчивости распределённой системы посредством преднамеренного внесения отказов в её компоненты и наблюдения за поведением системы под отказами.
- Consistent Hashing — семейство алгоритмов, минимизирующих перераспределение ключей между узлами при изменении состава кластера с помощью хеширования.
- Control Plane, CPL — компонент, который отвечает за уровень управления: хранение конфигураций, размещение целевых групп и координацию узлов плоскости данных.

Data Plane, DPL	— уровень обработки сетевых пакетов на котором осуществляется непосредственная балансировка трафика.
DSR (Direct Server Return)	— схема, при которой ответный трафик от сервера обработки трафика направляется клиенту напрямую, минуя балансировщик нагрузки.
eBPF (extended Berkeley Packet Filter)	— технология безопасного исполнения программ внутри ядра Linux, применяемая для сетевой обработки, наблюдаемости и фильтрации.
ECMP (Equal-Cost Multi-Path)	— механизм маршрутизации, позволяющий распределять трафик между несколькими равнозначными по стоимости путями.
Endpoint	— пара IP-адреса клиентского сервера и порта, идентифицирующая конкретный сервер обработки трафика в составе целевой группы балансировщика.
etcd	— распределённое отказоустойчивое хранилище типа "ключ-значение", обеспечивающее линейризуемые операции и используемое для координации и хранения конфигурации.
Gossip/SWIM	— семейство полностью децентрализованных протоколов обнаружения и мониторинга членства

узлов в распределённом кластере, основанных на случайном обмене сообщениями между узлами.

gRPC — фреймворк удалённого вызова процедур с двоичной сериализацией и поддержкой потоковых вызовов, применяемый для взаимодействия плоскости управления и узлов плоскости данных.

Health Check, HC — процесс периодического опроса эндпоинтов с целью определения их работоспособности.

Kernel bypass — подход к высокопроизводительной сетевой обработке, при котором пакеты передаются между сетевой картой и пользовательским пространством, минуя сетевой стек ядра.

L3/L4 — сетевой и транспортный уровни эталонной модели OSI, на которых работают рассматриваемые в работе балансировщики нагрузки, оперирующие IP-адресами и портами без анализа прикладных данных.

LBaaS (Load Balancer as a Service) — модель предоставления балансировщика нагрузки как управляемого сервиса в облаке.

Long-poll — техника взаимодействия клиента и сервера, при которой сервер удерживает запрос до появления новых данных или истечения заданного таймаута.

Placement	— отображение множества целевых групп на конкретные узлы плоскости данных.
Reconciliation	— процесс приведения фактического состояния системы к желаемому, в результате которого исполнитель применяет необходимые изменения и фиксирует достигнутый результат.
Replication Factor	— число копий целевой группы или vshard, размещаемых на различных узлах для обеспечения отказоустойчивости.
RIP (Real IP)	— реальный IP-адрес конкретного узла обработки трафика.
SLA (Service Level Agreement)	— соглашение об уровне обслуживания, определяющее гарантии доступности и производительности сервиса.
StatefulSet	— тип рабочей нагрузки в Kubernetes, обеспечивающий стабильные сетевые идентификаторы и упорядоченное развёртывание для приложений с состоянием.
Target Group, TG	— логическая группа эндпоинтов, объединённых общим VIP, портом и протоколом транспортного уровня (TCP или UDP).
VIP (Virtual IP)	— виртуальный IP-адрес, предоставляемый балансировщиком для приёма пользовательского

трафика и позволяющий абстрагировать клиента от конкретных узлов обработки.

VPP (Vector Packet Processing) — высокопроизводительный фреймворк обработки сетевых пакетов с применением векторной обработки и kernel bypass.

Vshard — промежуточная единица шардирования, используемая при двухуровневом консистентном хешировании для распределения эндпоинтов между узлами кластера проверок состояния.

XDP (eXpress Data Path) — программируемый сетевой механизм Linux на основе eBPF, позволяющий обрабатывать пакеты на этапе их поступления в драйвер сетевой карты, до основной части сетевого стека.

ВВЕДЕНИЕ

В настоящее время облачные технологии стали неотъемлемой частью инфраструктуры современного бизнеса. Развитие облачных платформ привело к массовому переносу сервисов в облачную среду, от веб-приложений и систем обработки финансовых транзакций до научных и инженерных вычислений. Особенно ярко этот тренд проявляется в области машинного обучения, где обучение и эксплуатация больших языковых моделей требуют крупных GPU-кластеров, самостоятельная закупка и сопровождение которых обходятся в десятки миллионов долларов и предполагают узкоспециализированные инженерные компетенции. Для большинства компаний аренда таких мощностей у облачного провайдера остаётся единственным экономически оправданным решением. Беря на себя эксплуатацию клиентской инфраструктуры, провайдер принимает и обязательства по её доступности, которые закрепляются в соглашениях об уровне обслуживания (SLA) на уровне 99,95–99,99 % времени. Нарушение этих гарантий приводит к ощутимым финансовым и репутационным потерям как для самого провайдера, так и для его клиентов, пользователи которых теряют доступ к зависящим от облака сервисам. Так, по оценкам ITIC, для большинства средних и крупных предприятий час простоя ИТ-систем может обходиться дороже 300 тыс. долларов, а для значительной их доли превышать 1 млн [1].

Для соблюдения подобных гарантий доступности облачный провайдер опирается на набор архитектурных механизмов, одним из ключевых среди которых является сетевая балансировка нагрузки. Балансировщик предоставляет единый виртуальный IP-адрес (VIP), за которым скрывается множество серверов обработки трафика, и автоматически перенаправляет запросы только на работоспособные узлы. Это позволяет клиенту развернуть свой сервис на нескольких виртуальных

машинах без самостоятельной реализации логики балансировки и сохранять работоспособность при выходе из строя одного или нескольких узлов обработки. Помимо отказоустойчивости, балансировка обеспечивает горизонтальную масштабируемость, поскольку добавление новых узлов увеличивает общую пропускную способность системы без изменения точки входа для клиентского трафика. Дополнительно достигается снижение задержек за счёт распределения нагрузки между менее нагруженными узлами, повышается утилизация ресурсов, а на стороне самого балансировщика становится возможной фильтрация и ограничение частоты запросов.

Сетевая балансировка задействована не только в обслуживании внешнего пользовательского трафика. Внутренние сервисы облака, такие как базы данных, очереди сообщений и сервисы аутентификации, практически всегда работают за сетевым балансировщиком, обеспечивающим отказоустойчивость уже межсервисного взаимодействия. Таким образом, балансировщик становится компонентом, через который проходит значительная часть как внешнего, так и внутреннего трафика облака, а его недоступность или некорректное поведение напрямую сказывается на доступности клиентских сервисов и внутренней инфраструктуры провайдера.

Балансировка нагрузки может осуществляться на различных уровнях модели OSI. Балансировщики уровня L7 (прикладной уровень) работают с содержимым HTTP-запросов, заголовками и другими прикладными данными, что позволяет принимать более качественные решения о маршрутизации. Однако это сопряжено с существенными накладными расходами на разбор, анализ и поддержание состояний протоколов высокого уровня. В облачной среде, где требуется обрабатывать миллионы пакетов в секунду при минимальных задержках, такие накладные расходы

зачастую неприемлемы. Балансировщики же уровня L3/L4 (сетевой и транспортный уровни) работают с IP-адресами и портами, не погружаясь в содержимое пакетов, что делает их значительно производительнее L7 решений. Именно поэтому в качестве основы облачной балансировки нагрузки используются L3/L4-решения, а L7-балансировка, как правило, реализуется отдельным слоем поверх них.

Помимо этого, крупные облачные провайдеры, такие как: Google, Amazon, Yandex уже имеют собственные реализации L3/L4 балансировщиков, однако детали их реализации не разглашаются. Наиболее подробно описанной и качественной работой на тему L3/L4 балансировщиков нагрузки является Google Maglev [2], архитектура которого была опубликована в 2016 году. Однако, его исходный код закрыт и привязан к внутренней инфраструктуре Google. В открытом доступе существуют лишь отдельные компоненты, решающие часть задачи. Например, Katran [3] от Meta реализует высокопроизводительную плоскость данных на базе eBPF, а IPVS [4] предоставляет встроенную в ядро Linux балансировку. Тем не менее ни одно из существующих открытых решений не покрывает все требования к облачному L3/L4 балансировщику, что будет более подробно разбираться в разделе 1.4. Решения, встроенные в экосистему Kubernetes [5] (MetalLB, kube-proxy), также не подходят для рассматриваемого случая, поскольку целевая платформа не является Kubernetes-кластером и имеет собственную архитектуру управления виртуальными машинами и сетью. Помимо этого, реализация балансировщика существенно зависит от специфики конкретного облачного провайдера — его сетевой топологии, используемых технологий виртуализации и требований к интеграции с другими сервисами, что делает невозможным прямое заимствование чужих решений без глубокой адаптации. Ввиду всего вышесказанного была поставлена задача создания

достаточно обобщенного решения, не завязанного на конкретные проприетарные компоненты инфраструктуры, что дополнительно ограничивало выбор готовых инструментов.

Целью работы является реализация отказоустойчивого многопользовательского L3/L4 балансировщика нагрузки на базе VPP, адаптированного под нужды облачного провайдера. Для достижения данной цели были поставлены следующие задачи:

- проектирование архитектуры целевого решения, выбор алгоритмов и технологий, которые лягут в его основу;
- проектирование и реализация модуля мониторинга состояния клиентских приложений;
- реализация плоскости управления, ответственной за хранение конфигураций, размещение целевых групп по узлам плоскости данных и предоставление API;
- реализация плоскости данных, позволяющей приводить состояние балансировщика на узле к целевому состоянию;
- функциональное и нефункциональное тестирование реализованного решения, включая хаос-тестирование;
- интеграция разработанных компонентов в облако.

В первом разделе данной работы освещены существующие решения в области балансировки нагрузки, проведен их сравнительный анализ и сформулированы требования к собственному решению. Во втором разделе предложена и описана общая архитектура системы, включая модель данных, паттерн реконсиляции и основные потоки данных. Третий раздел посвящён VPP как основе плоскости данных и обоснованию его выбора. В четвертом разделе рассмотрена реализация агента плоскости данных — компонента, ответственного за приведение фактического состояния балансировщика к желаемому. Пятый раздел описывает плоскость

управления — центральный компонент управления системой. Шестой раздел посвящён децентрализованной системе проверок состояния серверов обработки трафика. Седьмой раздел содержит результаты функционального, нагрузочного и хаос-тестирования, подтверждающие отказоустойчивость и работоспособность реализованного решения.

1 Обзор существующих решений

Подробные технические описания промышленных L3/L4 балансировщиков встречаются в открытом доступе редко. Большинство крупных облачных провайдеров рассматривают свои реализации как часть конкурентного преимущества и не публикуют деталей, ограничиваясь маркетинговыми материалами и описаниями API. Из систем, по которым доступны достаточно полные технические публикации, в настоящем разделе подробно разобраны Google Maglev, Cloudflare Unimog и L3/L4 балансировщик Яндекс Облака. На основе этого разбора и характеристик целевой облачной платформы в подразделе 1.4 сформулированы требования к L3/L4 балансировщику нагрузки в облачной среде.

1.1 Google Maglev

Исторически первым и наиболее детально описанным в открытой литературе промышленным программным L3/L4 балансировщиком нагрузки является Maglev, разработанный в Google. Эта система, описанная в работе Eisenbud и соавторов в 2016 году, обеспечивает обработку всего входящего трафика сервисов компании и стала прообразом для последующих разработок в данной области [2].

Архитектура Maglev принципиально отличается от классических схем active-standby, применявшихся ранее. В отличие от топологии с выделенным активным узлом и резервным, в Maglev используется ферма равноправных машин, между которыми трафик распределяется посредством ECMP (Equal-Cost Multi-Path) с маршрутизаторов перед ними. Каждая машина фермы способна обрабатывать любой пакет независимо от остальных, что исключает единую точку отказа на уровне балансировки [2].

Также в подходе active-standby резервный узел в штатном режиме не нагружается, а это расточительно для систем L3/L4 балансировки,

требовательных к производительности оборудования. Вдобавок, переключение между активным и резервным узлами нередко сопровождается разрывом существующих соединений, что плохо влияет на SLI балансировщика и клиента. В ферме Maglev каждый узел непрерывно обрабатывает трафик, а при выходе из строя нескольких машин маршрутизатор автоматически перестаёт направлять на них пакеты, без какого-либо вмешательства оператора [2].

Производительность системы обеспечивается применением kernel bypass. Сетевой интерфейс передаёт пакеты напрямую в пользовательское пространство, обходя сетевой стек ядра. Согласно разделу 5.2 статьи [2], при работе на восьмиядерном сервере с интерфейсом 10 Гбит/с Maglev достигает пиковой пропускной способности около 12 миллионов пакетов в секунду при пяти потоках обработки, после чего узким местом становится NIC. Ядерный стек Linux в аналогичных условиях показывает менее 30% этого результата.

Также в качестве ключевого алгоритмического вклада разработчики предложили алгоритм консистентного хеширования Maglev Hash, специально адаптированный для задачи привязки соединений к узлам обработки. Для каждого соединения вычисляется 5-tuple хеш, который отображается на конкретный узел фермы. Благодаря этому, при добавлении или удалении узла перестраивается только часть хеш-таблицы, что минимизирует количество разрываемых соединений [2].

Следует отметить поддержку режима Direct Server Return в архитектуре Maglev. В этом режиме ответный трафик от узла обработки возвращается клиенту напрямую, минуя балансировщик нагрузки. Это особенно важно, поскольку объём исходящего трафика в типичных веб-приложениях многократно превышает входящий. Благодаря DSR балансировщик обрабатывает только входящие пакеты, что позволяет

масштабироваться горизонтально без линейного роста требований к пропускной способности [2].

Система является проприетарной, её исходный код закрыт, а архитектура привязана к внутренней инфраструктуре Google. Однако опубликованные технические решения оказали определяющее влияние на проектирование данной работы. Из статьи заимствованы три принципа, а именно ферма равноправных узлов с ЕСМР для исключения единой точки отказа, kernel bypass для высокопроизводительной обработки пакетов и консистентное хеширование для аффинности соединений.

1.2 Cloudflare Unimog

Ещё одним промышленным решением в области L4-балансировки нагрузки является Unimog, разработанный компанией Cloudflare и описанный в 2020 году. В отличие от Maglev, который был спроектирован для дата-центров с контролируемой инфраструктурой, Unimog создавался для работы в граничной сети Cloudflare, где сотни точек присутствия по всему миру обрабатывают трафик максимально близко к конечным пользователям [6].

Эта разница в предназначении накладывает существенные ограничения на архитектуру системы. Например, в граничной сети нагрузка на серверы внутри одной точки присутствия может быть крайне неравномерной из-за изменения паттернов пользовательского трафика, географического распределения запросов и различий в популярности контента. Статическое консистентное хеширование, применяемое в Maglev, здесь оказывается недостаточным. Даже при равномерном распределении соединений реальная загрузка процессоров может существенно различаться.

Создатели Unimog реализовали динамическое перебалансирование нагрузки. Система постоянно отслеживает текущую загрузку каждого сервера и способна перенаправить пакет на менее загруженный узел. Для этого серверы обмениваются информацией о своём состоянии с соседями, что позволяет принимать решения о маршрутизации с учётом реальной доступности ресурсов [6].

У этого подхода есть ограничения, критичные для облачного сценария. Динамическое перебалансирование увеличивает сложность системы и нарушает принадлежность соединения конкретному узлу. При изменении загрузки пакеты одного соединения могут быть направлены на разные узлы обработки. Требование из раздела 1.4 о направлении всех пакетов одного соединения на один узел в этом случае не выполняется без дополнительных механизмов синхронизации состояния. Кроме того, Unimog предполагает прямой доступ балансировщика к метрикам загрузки узлов обработки, чего в облачном сценарии нет. Эти узлы принадлежат пользователю и для мониторинга со стороны балансировщика недоступны.

Как и Maglev, Unimog является проприетарной системой с закрытым исходным кодом. Детали реализации алгоритмов динамического перебалансирования, обмена состоянием и обработки пограничных случаев не опубликованы.

1.3 Yandex Cloud L3/L4 Load Balancer

Ещё одним примером промышленной реализации L3/L4 балансировщика нагрузки является решение, разработанное в Яндекс Облаке [7]. Яндекс опубликовал достаточно подробное описание архитектуры, что нетипично для крупного провайдера и позволяет провести содержательный анализ.

В плоскости данных Яндекс применяет VPP (Vector Packet Processing) для высокопроизводительной обработки пакетов [8]. Управляющая плоскость декомпозирована на несколько компонентов, а именно распределённое хранилище для хранения состояния системы, управляющую службу для конфигурации и размещения виртуальных адресов и отдельный сервис проверок состояния узлов обработки.

Сервис проверок разделён на управляющий API и исполнительные узлы, непосредственно опрашивающие серверы. Такое разделение позволяет масштабировать исполнительный слой независимо от управляющей плоскости. При росте числа проверяемых серверов добавляются исполнительные узлы, тогда как управляющий API остаётся единым для всей системы. Выход из строя одного компонента не приводит к полной недоступности балансировщика, а ограничивается деградацией отдельных функций.

Детали внутреннего устройства кластера проверок в публикации не раскрыты. Нет описания того, как шардируются проверки между исполнительными узлами, как обнаруживаются отказы этих узлов и как перераспределяются проверки при изменении состава кластера. Без решения этих задач исполнительные узлы проверок образуют единую точку отказа.

Как и другие решения в данной области, система Яндекс Облака является проприетарной и привязана к инфраструктуре провайдера. Публикация подтверждает тезис о том, что многие действительно крупные облачные провайдеры строят собственный балансировщик, аналоги которого в открытом доступе отсутствуют.

1.4 Требования к целевому решению

Для систематического сравнения существующих подходов и последующего проектирования собственной системы необходимо зафиксировать набор требований, которым должен удовлетворять L3/L4 балансировщик нагрузки в облачной среде. Данные требования были сформированы на основе анализа публичных описаний архитектуры Google Maglev [2], опыта построения подобных систем, изложенного Яндексом [7], SLA облачных провайдеров [9], а также практических потребностей целевой платформы.

Функциональные требования определяют набор возможностей, которые должны быть предоставлены пользователям балансировщика:

- Виртуальные IP-адреса (VIP). Балансировщик должен предоставлять клиентам облака виртуальные IP-адреса, за каждым из которых скрывается набор узлов обработки трафика. VIP служит единой точкой входа для клиентского трафика и не привязан к конкретной физической или виртуальной машине. Это упрощает масштабирование и не требует от клиента знания о фактическом размещении узлов.

- Консистентность соединений. Пакеты, принадлежащие одному TCP- или UDP-соединению, должны направляться на один и тот же узел обработки на протяжении всего времени жизни соединения. Для этого необходим механизм привязки на основе 5-tuple хеша. Нарушение этого требования приводит к разрыву активных соединений при изменении состава узлов обработки трафика;

- Активные проверки состояния узлов обработки. Балансировщик должен направлять трафик только на работоспособные узлы. Для этого необходимы периодические проверки доступности с настраиваемой периодичностью и порогом обнаружения отказа. Минимально необходимы два уровня проверок. На транспортном уровне проверка состоит в

установлении TCP-соединения и достаточна для большинства сервисов, у которых готовность к обслуживанию совпадает с готовностью принять подключение. На прикладном уровне проверка выполняется через HTTP-запрос к заданному пути и применима для сервисов, у которых TCP-порт открывается до фактической готовности отвечать на запросы. Именно вопрос организации проверок состояния представляет собой наиболее сложную задачу при проектировании облачного балансировщика. Требование отказоустойчивости означает, что система проверок не должна иметь единой точки отказа, а горизонтальная масштабируемость накладывает требование шардирования проверок между множеством узлов. Поэтому ни одно из готовых решений не покрывает этот компонент целиком, и его собственная разработка описана в разделе 6;

— Программный API для управления конфигурацией. Создание, изменение и удаление виртуальных IP, узлов обработки и параметров проверок состояния должны выполняться через программный интерфейс, а не вручную. Это необходимо для интеграции с системами оркестрации облака и для предоставления балансировки как услуги конечным пользователям;

— Доступ к исходному коду и возможность его модификации. Реализация L3/L4 балансировщика существенно зависит от специфики конкретного облачного провайдера, а именно его сетевой топологии, применяемых технологий виртуализации и требований к интеграции с остальными сервисами платформы. Поэтому целевое решение должно допускать адаптацию под инфраструктуру провайдера, что возможно лишь при наличии открытого и модифицируемого исходного кода. Проприетарные системы, описанные только на уровне архитектуры и привязанные к внутренней инфраструктуре своих разработчиков, этому требованию не удовлетворяют независимо от качества заложенных в них

решений, тогда как доступные открытые компоненты предоставляют исходный код, но покрывают лишь часть перечисленных требований. Данное требование носит характер предварительного условия, поскольку определяет не поведение системы во время работы, а саму возможность построения и эксплуатации сервиса балансировки облачным провайдером на её основе;

— Декларативная модель конфигурации. Клиент описывает желаемое состояние балансировщика, а система самостоятельно приводит к нему фактическое состояние и поддерживает его при сбоях. Декларативный подход обеспечивает идемпотентность повторных запросов, упрощает интеграцию с системами управления инфраструктурой как кодом и делает конфигурацию воспроизводимой. Этот же принцип впоследствии используется внутри системы как механизм отказоустойчивости (раздел 2.3), однако его прямая ценность для пользователя, простота управления, самостоятельна.

Нефункциональные требования определяют характеристики системы, критичные для работы в облачной среде:

— Высокая производительность плоскости данных. Балансировщик должен обрабатывать миллионы пакетов в секунду на одном узле при задержках порядка единиц микросекунд. Это исключает решения, работающие через стандартный сетевой стек ядра без оптимизаций, и накладывает требование на использование технологий `kernel bypass`;

— Отказоустойчивость без единой точки отказа. Выход из строя любого отдельного компонента системы (узла плоскости данных, экземпляра управляющей плоскости или узла выполнения проверок состояния) не должен приводить к недоступности балансировки для пользователей, чтобы удовлетворять SLA;

— Горизонтальная масштабируемость. Увеличение количества обслуживаемых VIP, узлов обработки трафика или объёма входящего трафика должно достигаться добавлением новых узлов в соответствующие компоненты, а не наращиванием мощности существующих серверов. Все компоненты архитектуры должны поддерживать шардирование и репликацию. При этом система должна функционировать как при развёртывании в Kubernetes, так и в классических окружениях с виртуальными машинами, что исключает жёсткую зависимость от механизмов обнаружения сервисов Kubernetes;

— Поддержка узлов обработки в сетях, не контролируемых оператором балансировщика. Архитектура должна допускать размещение узлов обработки трафика в сетях, недоступных для прямого управления балансировщиком, например в чужих облаках или за сетевыми границами. Балансировщик должен уметь направлять трафик на внешние хосты и выполнять проверки их состояния через публичные сети;

— Поддержка Direct Server Return (DSR). Ответный трафик от узла обработки должен возвращаться клиенту напрямую, минуя балансировщик. Это снимает с узлов балансировки нагрузку по обработке исходящего трафика, объём которого, как правило, многократно превышает входящий. Реализация DSR требует специальной инкапсуляции пакетов на сетевом уровне.

Дальше эти требования используются как критерии при оценке существующих решений и при проектировании собственной системы.

1.5 Выводы

Проведённый обзор существующих систем L3/L4 балансировки нагрузки демонстрирует отсутствие полноценных открытых решений, которые могли бы быть непосредственно использованы облачным

провайдером. Каждый из рассмотренных проектов решает определённое подмножество задач, однако ни один не покрывает всех формулируемых требований.

Google Maglev задаёт эталонную архитектуру программного балансировщика, а именно ферму равноправных узлов с ECMP, kernel bypass для высокой производительности, консистентное хеширование для связности соединений и DSR. Однако система является проприетарной, привязанной к внутренней инфраструктуре Google и недоступной для воспроизведения вне компании.

Cloudflare Unimog демонстрирует альтернативный подход с динамическим перебалансированием, актуальный для граничных сетей с непредсказуемой нагрузкой. Для контролируемой среды централизованного дата-центра, характерной для облачного провайдера, архитектура Maglev оказывается более подходящей.

Yandex Cloud подтверждает ключевые архитектурные решения Maglev и демонстрирует практическую реализацию декомпозиции системы на независимые компоненты. Публикация показывает, как организовать проверки состояния и управляющую плоскость в соответствии с требованиями отказоустойчивости и масштабируемости.

Исследование решений для плоскости данных выявило три технологических направления. IPVS представляет собой зрелое, но ограниченное по производительности решение. Katran показывает промежуточные характеристики на основе XDP/eBPF. VPP обеспечивает максимальную производительность благодаря полному обходу ядра и векторной обработке. Дальнейшее обоснование выбора технологии для плоскости данных приведено в разделе 3.

Поэтому для реализации полноценного облачного L3/L4 балансировщика нагрузки необходимо собирать решение из компонентов,

проектировать собственную управляющую плоскость и систему проверок состояния. Обзоры существующих подходов к этим компонентам приведены в начале соответствующих разделов настоящей работы.

2 Архитектура решения

Проведённый обзор существующих систем в предыдущем разделе показал отсутствие готовых открытых решений, удовлетворяющих всем сформулированным требованиям. В текущем разделе описана архитектура разработанного балансировщика в целом, затем, в последующих разделах каждый компонент будет рассматриваться более подробно.

2.1 Компонентная архитектура

Разработанная система состоит из плоскости управления, системы проверок состояния и агента плоскости данных. Такое разделение повышает отказоустойчивость, поскольку выход из строя одного компонента не приводит к полной недоступности всего балансировщика.

На рисунке 1 схематично представлена архитектура системы и взаимодействие её компонентов.

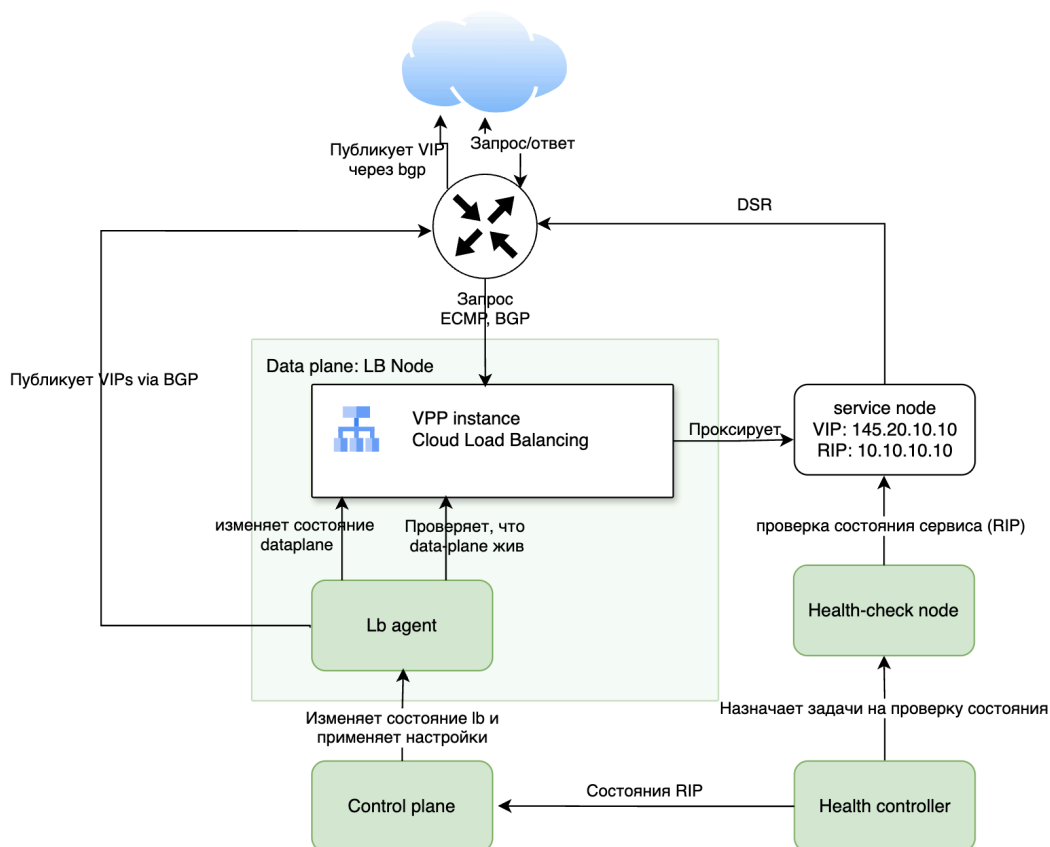


Рисунок 1 — Общая компонентная архитектура балансировщика:
плоскость управления, кластер проверок состояния и агенты плоскости
данных

Плоскость управления (control plane) централизованно хранит конфигурацию балансировщика и распределяет её по узлам плоскости данных. Она включает API-сервер для приёма запросов, реестр для хранения состояния и планировщик для размещения целевых групп по узлам. Таким образом плоскость управления отвечает за хранение и формирование желаемого состояния системы.

Система проверок состояния (health check node и health controller) выполняет активный опрос эндпоинтов и определяет их работоспособность. Она построена как децентрализованный кластер без единой точки отказа.

Множество узлов проверок координируются через gossip-протокол [10] и распределяют нагрузку через консистентное хеширование.

Наконец, агент плоскости данных (lb agent) запускается на каждом узле обработки трафика. Он получает конфигурацию для плоскости данных на узле от плоскости управления, статусы здоровья от системы проверок и приводит фактическое состояние плоскости данных к желаемому. Агенты не зависят друг от друга и работают независимо.

Можно заметить, что по структуре это тот же паттерн, что и в Kubernetes, где API-сервер хранит состояние, контроллеры принимают решения, а kubelet приводит состояние узла к желаемому. Но разработанная система не является клоном Kubernetes и предназначена для работы как в самом Kubernetes, так и в классических окружениях с виртуальными машинами.

2.2 Модель данных

Центральным понятием системы является целевая группа (TargetGroup). Она объединяет спецификацию балансировки, список эндпоинтов и параметры проверок состояния. Спецификация балансировки задаёт протокол (TCP или UDP), порт и виртуальный IP-адрес. Параметры проверок описывают тип проверки (TCP, HTTP или GRPC), её периодичность, таймаут и порог обнаружения отказа. Каждая целевая группа идентифицируется уникальным идентификатором и принадлежит конкретному пользователю облака.

Размещение (Placement) определяет привязку целевых групп к узлам плоскости данных. Для обеспечения отказоустойчивости каждая целевая группа размещается одновременно на нескольких узлах согласно настраиваемому фактору репликации. Размещение хранит версию

конфигурации для каждого узла, что позволяет отслеживать актуальность применённой конфигурации.

Состояние узла плоскости данных (DataPlaneState) определяет его статус в системе и принимает значения alive (работоспособен), dead (недоступен), drained (выведен из эксплуатации) или unknown (неизвестен). Переходы между состояниями обрабатываются плоскостью управления с учётом задержек для предотвращения моргания.

Статус здоровья эндпоинта (EndpointStatus) принимает значения healthy и unhealthy и относится к конкретному эндпоинту в составе целевой группы. Статус определяется системой проверок состояния и доставляется агентам отдельным потоком, минуя плоскость управления. Дополнительно с каждым статусом в агенте ассоциируется поколение статусов целевой группы, которое монотонно возрастает при любом изменении здоровья хотя бы одного её эндпоинта в целевой группе и служит сигналом для запуска цикла реконсиляции агентом.

2.3 Паттерн реконсиляции

Ключевым архитектурным принципом, через который реализуется отказоустойчивость системы, является реконсиляция. Суть её в непрерывном приведении фактического состояния к желаемому, что обеспечивает автоматическое восстановление после сбоев без специальной обработки ошибок.

Практика применения декларативного управления в крупных распределённых системах была детально описана в работе Google о системе управления сетевой инфраструктурой Orion [11]. Авторы столкнулись с фундаментальной проблемой императивного подхода, при котором система управления напрямую манипулирует сетевыми устройствами. При этом любой временный сбой связи или перебой в работе контроллера приводит к

неконсистентному состоянию, а восстановление требует сложной логики повторных попыток. В результате в Ogiop был реализован другой подход, при котором система управления декларирует желаемое состояние, а локальные агенты непрерывно сравнивают фактическое состояние с желаемым и устраняют расхождения.

Этот же подход лег в основу архитектуры Kubernetes [12]. API-сервер хранит желаемое состояние кластера в etcd, а встроенные контроллеры непрерывно отслеживают текущее состояние ресурсов. При обнаружении расхождения между желаемым и фактическим состоянием контроллер инициирует операции по приведению системы к целевому состоянию. На каждом узле kubelet получает список подов, которые должны быть запущены, и обеспечивает соответствие фактического состояния узла этому списку. Это обеспечивает идемпотентность операций. Повторная подача того же желаемого состояния не приводит к побочным эффектам, а восстановление после прерывания не требует специальной обработки.

В разработанной системе реконсиляция применяется на двух уровнях. Плоскость управления выполняет глобальную реконсиляцию размещения целевых групп. Планировщик отслеживает изменения в составе целевых групп и узлов плоскости данных, вычисляет оптимальное размещение и приводит фактическое распределение к желаемому. В то же время каждый агент плоскости данных выполняет локальную реконсиляцию конфигурации балансировщика. Агент непрерывно синхронизирует состояние локального балансировщика с полученной конфигурацией и учитывает актуальные статусы здоровья эндпоинтов из очереди сообщений.

Выбор реконсиляции вместо императивного управления обусловлен особенностями распределённых систем. В таких системах сетевые задержки и временные недоступности компонентов являются нормой, а не исключением. Императивные команды требуют подтверждения исполнения

и сложной логики обработки ошибок при сбоях. В отличие от них реконсиляция естественным образом переживает временные недоступности. Как только связь восстанавливается, агент получает актуальное желаемое состояние и приводит к нему фактическое. Реконсиляция также упрощает восстановление после отказов агентов. Перезапущенный агент просто считывает текущее желаемое состояние из локального хранилища или через повторное подключение к плоскости управления и начинает его применять, не требуя сложной процедуры восстановления.

Детали глобальной реконсиляции рассмотрены в разделе 5.3, локальная реконсиляция агента описана в разделе 4.

2.4 Потоки данных

В системе можно выделить несколько ключевых потоков данных, обеспечивающих её функционирование.

Поток пользовательского трафика проходит через маршрутизатор, узлы плоскости данных и сервисные узлы. Маршрутизатор равномерно распределяет входящие соединения между доступными узлами плоскости данных. Каждый узел самостоятельно определяет целевой сервисный узел на основе хеша от 5-tuple, что гарантирует постоянство направления для всех пакетов одного соединения. Ответный трафик в режиме DSR направляется напрямую от сервисного узла к маршрутизатору, минуя плоскость данных.

Помимо пользовательского трафика есть поток управляющих данных, который идёт от плоскости управления к агентам плоскости данных. Плоскость управления отслеживает изменения в конфигурации целевых групп и их размещении, формирует различия для каждого агента и доставляет их через gRPC long-poll соединения. Агенты применяют

полученные изменения к локальным экземплярам VPP и подтверждают применение.

Также есть поток данных о состоянии эндпоинтов, который доставляется к агентам напрямую, минуя плоскость управления, через цепочку CDC, подробно описанную в разделе 6.5. Такое разделение потоков позволяет агентам получать обновления статусов даже при временной недоступности плоскости управления.

3 Решения для плоскости данных

В предыдущих разделах уже было выявлено, что полных открытых L3/L4 балансировщиков, покрывающих все функциональные требования облачной системы, не существует. Однако для компонента плоскости данных, непосредственно отвечающего за пересылку пакетов, доступен ряд специализированных открытых решений. Они существенно различаются по уровню обработки пакетов. Одни решения работают через стандартный сетевой стек ядра, другие используют специализированные фреймворки внутри ядра, третьи переносят обработку полностью в пользовательское пространство в обход ядра. Этот уровень в значительной степени определяет производительность и гибкость системы.

Рассмотрим доступные открытые решения для плоскости данных, их преимущества и ограничения.

3.1 IPVS

IPVS (IP Virtual Server) представляет собой наиболее зрелое и широко распространённое решение, встроенное в ядро Linux с 1999 года. Система реализована как модуль подсистемы netfilter и поддерживает несколько режимов работы, включая NAT, Direct Routing и туннелирование. Для распределения нагрузки IPVS предоставляет различные алгоритмы: round-robin, least connections, а также хеширование по адресу источника [4].

Благодаря многолетней эксплуатации IPVS обладает высокой стабильностью и хорошей документированностью. В частности, компонент kube-proxy в Kubernetes реализует абстракцию Service именно через IPVS при высоких нагрузках на сетевой стек кластера. Однако архитектура IPVS предполагает прохождение пакетов через подсистему netfilter ядра со всеми сопутствующими накладными расходами. Контекстные переключения, обработка прерываний и блокировки синхронизации ограничивают

производительность IPVS величиной порядка одного миллиона пакетов в секунду на одно ядро [2, 13].

3.2 Katran

Другое направление развития представлено проектом Katran, разработанным компанией Meta и опубликованным в 2018 году. Katran использует технологию eBPF (Extended Berkeley Packet Filter) для загрузки программы балансировки непосредственно в сетевой драйвер через интерфейс XDP (eXpress Data Path). Это позволяет перехватывать пакеты до их попадания в основной сетевой стек Linux и принимать решения о маршрутизации на ранней стадии обработки [3].

Также Katran реализует алгоритм Maglev Hash и поддерживает режим DSR, достигая производительности порядка десятков миллионов пакетов в секунду на одно ядро [3]. Однако XDP остаётся в рамках ядра операционной системы, что накладывает ряд ограничений. Программы eBPF имеют фиксированный набор инструкций и пределы по размеру и вложенности, а отладка и диагностика затруднены. Кроме того, константное выделение буферов в драйвере требует тщательной настройки под конкретное оборудование.

3.3 VPP

Третий подход реализован во фреймворке VPP (Vector Packet Processing), разрабатываемом в рамках проекта FD.io от Linux Foundation. В отличие от предыдущих решений, VPP полностью обходит сетевой стек ядра за счёт DPDK (Data Plane Development Kit) [14]. Сетевой адаптер передаёт пакеты напрямую в пользовательское пространство, что исключает системные прерывания, переключения контекста и копирование данных между адресными пространствами [8].

Наряду с обходом ядра VPP реализует векторную модель обработки пакетов. Вместо поштучной обработки каждого пакета система группирует их в векторы, которые последовательно проходят этапы обработки, такие как разбор заголовков, маршрутизация и инкапсуляция. Такой подход эффективно использует кэши процессора, поскольку инструкции и структуры данных каждого этапа загружаются один раз и применяются ко всему вектору пакетов. Также это позволяет применять SIMD-инструкции и минимизировать промахи процессорного кэша, что даёт существенный прирост производительности при больших объёмах трафика [8]. Как и Maglev, VPP не требует специализированного сетевого оборудования и работает на стандартных серверах с поддержкой DPDK.

Вдобавок, архитектура VPP построена на основе плагиновой модели. Обработка пакетов представлена в виде графа узлов, в который разработчик может добавлять собственные этапы обработки без модификации ядра фреймворка. Кроме того, в состав VPP уже входит плагин lb, реализующий балансировку нагрузки на основе 5-tuple консистентного хеширования. Плагин поддерживает инкапсуляцию GRE для организации Direct Server Return, а также предоставляет механизмы NAT44 для трансляции сетевых адресов.

В опубликованных материалах проекта VPP приводятся результаты тестирования производительности, демонстрирующие обработку порядка десяти миллионов пакетов в секунду на одном ядре процессора [8]. Эти показатели сопоставимы с характеристиками Maglev и превосходят решения на основе XDP и netfilter в сценариях с высокой плотностью пакетов [13].

Для взаимодействия с VPP из внешних компонентов разработаны клиентские библиотеки. В частности, GoVPP представляет собой реализацию на языке Go, предоставляющую привязки к binary API VPP [15].

Это упрощает интеграцию с агентом, написанным на Go. Существуют также примеры комплексных решений, такие как проект Ligato, демонстрирующий подходы к управлению конфигурацией VPP через агентную модель.

3.4 Сравнение и выбор решения

В качестве основы для плоскости данных был выбран VPP. Полный обход ядра и векторная обработка обеспечивают производительность, сопоставимую с показателями Maglev, а плагиновая архитектура позволяет расширять функциональность без модификации ядра фреймворка. Встроенная поддержка GRE-инкапсуляции и NAT44 упрощает реализацию DSR и других сетевых функций. Размещение компонентов в пользовательском пространстве, помимо производительности, упрощает отладку и интеграцию с остальными компонентами системы, написанными на Go.

У этого выбора есть и обратная сторона. Эксплуатация VPP требует поддержки DPDK, hugepages, выделенных под обработку сетевого трафика ядер процессора и снятия сетевого адаптера со стандартного стека ядра. На обычной машине разработчика и в стандартном CI VPP не запускается без существенной подготовки окружения. Поэтому в проекте плоскость данных скрыта за общим интерфейсом, и наряду с самим VPP реализовано ещё несколько вариантов, рассчитанных на разные сценарии (тестирование, локальный запуск, упрощённая среда без DPDK). Подробное описание этой абстракции и её реализаций приведено в разделе 4.2.

Все перечисленные решения, включая VPP, отвечают только за пересылку пакетов на основе заданных правил балансировки и не покрывают задач управления конфигурацией и мониторинга состояния серверов.

4 Агент плоскости данных

Перейдём к рассмотрению агента плоскости данных, запускаемого непосредственно на каждом узле обработки трафика. Он получает желаемое состояние от центральной плоскости управления и непрерывно приводит к нему фактическое состояние локального балансировщика. Желаемое состояние описывает набор целевых групп, размещённых на узле, а фактическое отражает конфигурацию балансировщика и список активных эндпоинтов в нём. В настоящем разделе подробно рассмотрены архитектура агента, принципы конкурентной обработки, абстракция плоскости данных, механизмы реконсильации и источники входных данных.

4.1 Архитектура агента

Центральная часть агента представляет собой набор рабочих потоков, каждый из которых отвечает за реконсильацию закреплённого за ним подмножества целевых групп. Закрепление выполняется на основе хеша от идентификатора группы, что гарантирует последовательную обработку всех операций для одной группы и устраняет необходимость в синхронизации при работе с её состоянием.

При этом очередь каждого рабочего потока ограничена по размеру. Если поток занят, а очередь уже содержит задачу реконсильации той же группы, добавлять ещё одну нецелесообразно, поскольку к моменту выполнения она будет устаревшей. Актуальное состояние уже отражено в ожидающей задаче. Также задачи дедуплицируются до момента постановки в очередь, что исключает создание идентичных запросов при всплесках нагрузки.

Для надёжности агент периодически выполняет принудительную полную реконсильацию всех целевых групп, известных из локального хранилища. Это особенно важно для статусов здоровья, так как в отличие

от конфигурационного потока, статусы не имеют монотонных версий, по которым можно было бы надёжно отследить пропуски, о чём подробнее сказано в разделе 6.

А в случае ошибки реконсиляция повторяется ограниченное число раз, после чего задача отбрасывается. Это предотвращает бесконечные циклы при устойчивых сбоях, а пропущенное расхождение будет исправлено следующей периодической реконсильацией.

4.2 Абстракция плоскости данных

Чтобы логика реконсильации не зависела от конкретного механизма пересылки пакетов, между реконсилером и реализацией плоскости данных введён общий интерфейс. Весь код реконсильации работает исключительно через этот интерфейс, поэтому замена одной реализации на другую не требует изменений в нём.

Таким образом в проекте реализовано четыре варианта интерфейса. Основной взаимодействует с процессом VPP через GoVPP binary API [15] и используется в промышленном окружении. Реализация на базе IPVS вызывает `ipvsadm` запуском внешнего процесса и применяется на стендах, где DPDK недоступен. Реализация на чистом Go (`testgovpp`) принимает реальный трафик и распределяет его по серверам обработки трафика через `weighted rendezvous hashing` на 5-tuple без обращения к ядру. Этот вариант используется в интеграционных и хаос-тестах для проверки сквозного прохождения трафика. Заглушка же эмулирует успешное выполнение операций без работы с сетью и применяется при модульном тестировании логики реконсильации в изоляции.

4.3 Локальное хранилище

Наличие локального хранилища состояния является одним из принципиальных архитектурных решений агента. Агент сохраняет как

желаемое, так и фактическое состояние во встраиваемой базе данных, расположенной непосредственно на узле. Благодаря этому при временной недоступности плоскости управления реконсиляция продолжается в обычном режиме, поскольку последнее известное желаемое состояние остаётся доступным локально. При перезапуске агент первым делом читает локальное хранилище и восстанавливает конфигурацию балансировщика, не ожидая переподключения к плоскости управления.

Помимо текущего состояния, список эндпоинтов целевой группы хранится в виде базового снимка и журнала изменений поверх него. При запросе обновлений агент передаёт плоскости управления текущие версии, а сервер возвращает только изменения начиная с известной агенту версии. Версии при этом выполняют двойную роль. Они формируют инкрементальное обновление и одновременно позволяют отслеживать отставание агента от актуального состояния. Подробнее об этом механизме сказано в разделе 5.

4.4 Реконсиляция спецификации

Реконсиляция спецификации целевой группы запускается при изменении её версии в желаемом состоянии. Агент извлекает из локального хранилища версию последней применённой спецификации и сравнивает её с желаемой. При совпадении операция пропускается без обращения к балансировщику, что обеспечивает идемпотентность при повторных вызовах реконсиляции.

При расхождении версий желаемая спецификация передаётся балансировщику. Обновление версии в локальном хранилище выполняется только после получения подтверждения успеха, что исключает ситуацию, при которой агент считает состояние применённым, тогда как оно фактически не было доставлено. При удалении целевой группы из

желаемого состояния её конфигурация удаляется из балансировщика, а связанные записи очищаются из хранилища. В случае сбоя между этими двумя шагами следующая полная реконсиляция обнаружит расхождение и исправит его.

4.5 Реконсиляция эндпоинтов

Реконсиляция эндпоинтов имеет два независимых источника изменений. Первый представлен версией списка эндпоинтов, монотонно возрастающей при каждом добавлении или удалении. Второй связан с понятием поколения группы, которое инкрементируется при любом обновлении статуса здоровья, даже если сам список эндпоинтов остался прежним. Разделение этих двух сигналов необходимо, поскольку изменение статуса здоровья не затрагивает желаемый состав эндпоинтов, однако должно немедленно отражаться на том, какие из них активны в балансировщике.

Сравнение желаемого и фактического состояний выполняется с учётом актуальных статусов здоровья. Нездоровые эндпоинты не включаются в список добавляемых. Если нездоровый эндпоинт уже присутствует в балансировщике, он подлежит удалению. Эндпоинты, отсутствующие в желаемом состоянии, удаляются независимо от их статуса здоровья. Идемпотентность обеспечивается версионированием статусов. Повторная доставка статуса с уже известной версией не изменяет поколение группы и не инициирует лишнюю реконсиляцию.

4.6 Получение конфигурации от плоскости управления

Желаемое состояние доставляется агенту через gRPC по протоколу long-poll, детали серверной стороны которого рассмотрены в разделе 5. Агент устанавливает соединение, отправляет запрос с идентификатором узла, текущей версией размещения и версиями каждой из известных ему

целевых групп, после чего ожидает ответа. Плоскость управления использует переданные версии для формирования инкрементального обновления и возвращает только изменения, которые агент ещё не видел.

Сервер удерживает запрос до появления изменений либо до истечения таймаута. Получив ответ, агент сохраняет обновления в локальном хранилище, ставит в очередь задачи реконсиляции изменённых групп и немедленно устанавливает новое соединение. При разрыве агент продолжает работать с последним известным состоянием из локального хранилища.

Регистрация агента в системе выполнена через аренду etcd. При старте агент создаёт аренду и периодически её продлевает, а плоскость управления отслеживает активные аренды для определения живых узлов. Истечение аренды без продления служит признаком недоступности узла и запускает реконсиляцию размещения. Выбор etcd в этой роли обоснован в разделе 5.

4.7 Получение статусов здоровья

Статусы здоровья эндпоинтов поступают к агенту по пути, полностью независимому от плоскости управления. Система проверок доставляет их через цепочку CDC, описанную в разделе 6.5. Каждый агент потребляет сообщения с уникальным идентификатором группы, соответствующим идентификатору узла, и получает полную копию топика независимо от остальных.

Гарантия доставки в этом канале соответствует семантике *at-least-once*, то есть сообщение точно будет доставлено, но доставка может произойти более одного раза. Дублирование нейтрализуется идемпотентной обработкой на стороне агента, при которой повторный одинаковый статус просто игнорируется. При изменении статуса инкрементируется поколение соответствующей целевой группы, что инициирует реконсиляцию её

эндпоинтов. Поколение таким образом служит мостом между потоком статусов здоровья и циклом реконсиляции.

Кроме того, устойчивость к временной недоступности агента обеспечивается хранением сообщений в топике с настраиваемым сроком жизни. Пока агент недоступен, статусы накапливаются в топике, и после восстановления агент дочитывает все пропущенные события.

5 Плоскость управления

В предыдущем разделе был рассмотрен агент плоскости данных, получающий желаемое состояние извне и приводящий к нему локальный балансировщик. Настоящий раздел посвящён тому, откуда это состояние берётся. Речь идёт о плоскости управления. Этот компонент хранит конфигурацию всей системы, распределяет целевые группы по узлам и доставляет изменения агентам.

5.1 Обзор подходов к плоскости управления

Задача централизованного управления распределёнными компонентами хорошо изучена в индустрии. В сервисных сетях, например Istio, выделяется центральный управляющий компонент istiod, который хранит конфигурацию и доставляет её агентам (Envoy-прокси) по протоколу xDS. Протокол xDS предполагает инкрементальную доставку, при которой агент сообщает свою текущую версию ресурсов, а сервер возвращает только изменения. Аналогичная структура прослеживается в Linkerd, где destination-сервис выполняет роль централизованного источника маршрутной информации для прокси-агентов.

Помимо сервисных сетей, Kubernetes [5] реализует схожий паттерн на уровне управления рабочими нагрузками. Желаемое состояние кластера хранится в etcd, контроллеры подписываются на изменения через watch-операции и reconcile-циклом приводят фактическое состояние к желаемому. Каждый узел управляется kubelet-ом, который получает конфигурацию от API-сервера и самостоятельно применяет её локально. Для координации нескольких экземпляров контроллера используется выбор лидера через механизм аренды etcd.

Из этих систем были заимствованы несколько концептуальных решений, а именно центральное хранилище, планировщик и агенты, а также

разделение желаемого и фактического состояния. При этом разработанная плоскость управления не является Kubernetes и не зависит от него. Она реализует собственную модель данных и собственный алгоритм размещения, ориентированные на задачи балансировщика, а не на управление контейнерными рабочими нагрузками.

5.2 Компоненты плоскости управления

На рисунке 2 схематично представлены компоненты плоскости управления и их взаимодействие с агентом плоскости данных.

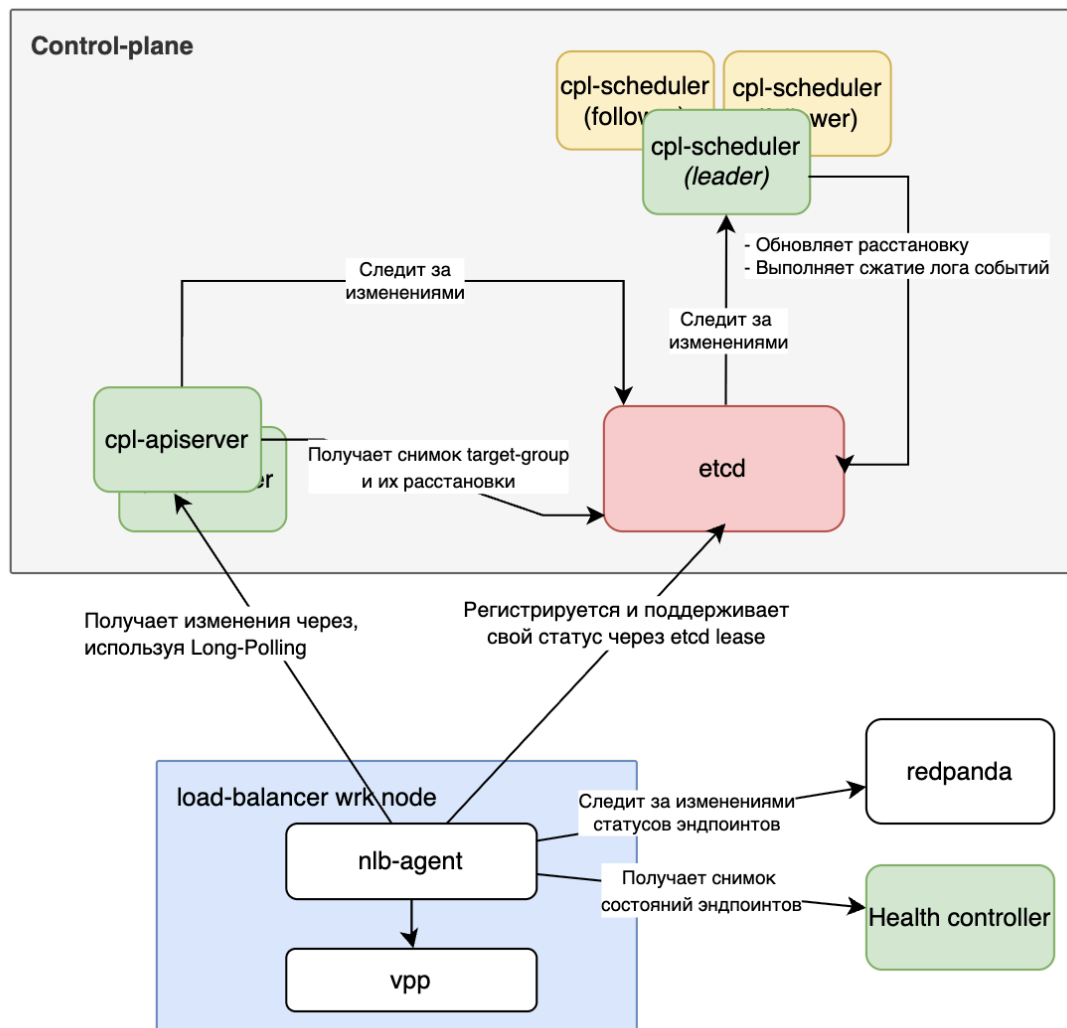


Рисунок 2 — Компонентная архитектура плоскости управления и схема взаимодействия с агентом плоскости данных

В целом разработанная плоскость управления состоит из двух самостоятельных процессов и разделяемого хранилища. API-сервер (cpl-apiserver) принимает от клиентов gRPC-запросы на создание, изменение и удаление целевых групп, а также соединения от агентов для получения конфигурации. Реконсиллятор (cpl-scheduler) работает как отдельный процесс, он получает события об изменениях в конфигурации и составе узлов и обновляет размещение целевых групп. Оба компонента взаимодействуют через etcd, которое служит единственным источником истины о состоянии всей системы.

Кроме постоянного хранилища, API-сервер поддерживает в памяти два кеша. Первый отражает текущее состояние узлов плоскости данных, второй хранит состояние целевых групп. Эти кеши формируются на основе watch-подписок на etcd и позволяют отвечать на запросы агентов без обращения к диску. На основе этих кешей работает система подписок, в рамках которой каждый агент, ожидающий обновлений по протоколу long-poll, регистрирует свою подписку. А она уже срабатывает при появлении изменений.

Кроме того, разделение API-сервера и реконсиллятора на два процесса обусловлено соображениями отказоустойчивости. API-сервер обслуживает как входящие запросы от клиентов, так и исходящие соединения от агентов, поэтому его недоступность немедленно сказывается на доставке конфигурации. Реконсиллятор же выполняет фоновые расчёты размещения и может кратковременно отсутствовать без потери конфигурации, поскольку всё необходимое уже записано в etcd. Это разделение тем более существенно, что логика планирования размещения нетривиальна, потребляет процессорное время в моменты массовых перестановок и, как любая сложная вычислительная компонента, имеет ненулевую вероятность отказа из-за необработанной ошибки или нехватки ресурсов. Изоляция этой

логики в отдельном процессе гарантирует, что её падение не затронет приём клиентских запросов и доставку уже рассчитанной конфигурации. Если реконсилятор перезапускается, новый экземпляр считывает текущее состояние из `etcd` и продолжает работу с того момента, где остановился предыдущий.

5.3 Поток конфигурационных данных

Прежде всего, поток конфигурации начинается с клиентского обращения к `ctrl-api-server`, который записывает изменение в `etcd`. Реконсилятор, отслеживающий пространство ключей, реагирует на событие и при необходимости обновляет размещение, после чего API-сервер уведомляет ожидающее `long-poll` соединение агента. Поток статусов здоровья намеренно выведен в обход плоскости управления, что подробнее рассмотрено в разделе 6. Поток обнаружения узлов опирается на `etcd`-аренды. Агент создаёт аренду при старте и периодически продлевает её, а при истечении плоскость управления фиксирует выбытие узла через `watch`-подписку.

Такое разделение потоков не случайно. Статусы здоровья меняются значительно чаще конфигурационных событий, и направление этого потока через плоскость управления нагроулило бы её без какой-либо необходимости. Кроме того, разделение повышает независимость компонентов. Агенты получают обновления статусов даже при временной недоступности плоскости управления.

5.4 Хранилище состояния

Выбор хранилища для конфигурации балансировщика предопределяет возможности всей плоскости управления. Рассматривались три основных варианта и их производные, а именно реляционная база

данных (PostgreSQL), хранилище типа ключ-значение с сильной согласованностью (etcd) и Redis.

PostgreSQL обладает богатой функциональностью, включая транзакции, индексы и сложные запросы, однако для данной задачи это скорее избыточность, чем преимущество. К тому же механизм реактивного оповещения LISTEN/NOTIFY значительно слабее нативных watch-операций etcd. Он не гарантирует доставки при разрыве соединения и не несёт версии изменения. Реализация аренды для обнаружения живых агентов потребовала бы отдельной логики с фоновыми задачами. А атомарный инкремент с проверкой условия (compare-and-swap) не является первоклассной операцией в PostgreSQL и реализуется через явные транзакции с уровнем изоляции SERIALIZABLE, что усложняет код и снижает производительность под нагрузкой.

В отличие от PostgreSQL, Redis не имеет проблем с реактивностью. Операции pub/sub и keyspace notifications обеспечивают своевременную доставку изменений. Однако Redis по умолчанию не гарантирует строгой линейизуемости, поскольку при асинхронной репликации возможна потеря данных при сбое мастера. Для хранения конфигурации балансировщика, от корректности которой зависит направление трафика, это неприемлемо.

Перечисленных недостатков лишён etcd [16], разрабатывавшийся именно для хранения небольших объёмов конфигурационных данных с требованиями сильной согласованности, то есть для той же задачи, для которой Google создал Chubby [17], ставший прообразом целого класса распределённых систем координации. Kubernetes [5] использует etcd по той же причине, для хранения желаемого состояния кластера, координации контроллеров через watch-подписки и выбора лидера через аренды. На базе алгоритма Raft [18] etcd гарантирует линейизуемые чтения и записи, что

делает условные транзакции первоклассным примитивом. Транзакция `etcd` позволяет атомарно прочитать и изменить набор ключей при выполнении заданных условий на их текущие значения или версии (конструкция `If/Then/Else`). На этом примитиве строятся как простые `compare-and-swap`-инкременты версий конфигурации, так и более сложные операции, согласованно затрагивающие несколько ключей сразу, например одновременное обновление спецификации целевой группы и инкремент её версии в отдельном ключе. `Watch`-подписки уведомляют подписчиков при любом изменении, доставляя событие вместе с его "ревизией", то есть порядковым номером изменения в журнале `Raft`. Механизм аренды встроен нативно. Агент создаёт аренду при старте, привязывает к ней ключ своего статуса и периодически продлевает её, а при истечении ключ удаляется автоматически, что `API`-сервер немедленно обнаруживает через `watch`.

Вместе с тем `etcd` образует единую точку отказа для всей плоскости управления. Сам `etcd` спроектирован как отказоустойчивый кластер. При трёх узлах он переносит выход из строя одного, при пяти допустим отказ двух. При полной недоступности `etcd` агенты продолжают работать автономно на основе последнего известного состояния из локального хранилища, а трафик не прерывается. По этим причинам в качестве хранилища выбран `etcd`.

Что касается технической реализации, пространство ключей `etcd` организовано следующим образом. Спецификации целевых групп размещены по пути `spec/desired/<tgID>`, а версия последнего подтверждённого состояния хранится по пути `spec/current/<tgID>`. Эндпоинты хранятся в двух представлениях, а именно компактный снимок в `endpoints/compacted/<tgID>` и журнал изменений в `endpoints/changelog/<tgID>`. Размещения по узлам лежат в `placements/<nodeId>`, статусы узлов размещены в `statuses/<nodeId>`. Такое

разделение позволяет подписываться на изменения отдельных аспектов конфигурации, не получая лишних событий.

5.5 Модель версионирования

Версионирование является одним из центральных механизмов плоскости управления и заслуживает отдельного рассмотрения.

Каждая целевая группа несёт две независимые версии. `SpecVersion` монотонно возрастает при каждом изменении спецификации, а именно протокола, порта или виртуального адреса. `EndpointsVersion` монотонно возрастает при каждом изменении состава эндпоинтов. Обе версии инкрементируются через `compare-and-swap` в `etcd`. Запись принимается только в том случае, если текущее значение совпадает с ожидаемым. Благодаря этому никакое параллельное изменение не может остаться незамеченным. Каждое успешное обновление строго увеличивает версию ровно на единицу.

Наряду с версиями целевой группы, версия размещения `PlacementVersion` ведётся отдельно для каждого узла плоскости данных и инкрементируется при каждом добавлении или удалении целевой группы на этом узле.

В совокупности эти версии решают несколько задач. Во-первых, они обеспечивают инкрементальную доставку конфигурации. Агент сообщает свои текущие версии, а сервер возвращает только изменения, произошедшие позже. При переподключении после длительного простоя агент получит ровно то, что пропустил, без полной перегрузки состояния. Во-вторых, версии гарантируют идемпотентность. Повторное применение конфигурации с той же версией не приводит к изменениям ни в агенте, ни в балансировщике. В-третьих, версии позволяют отслеживать отставание

каждого агента. Сравнив версию, известную агенту, с текущей версией в `etcd`, можно точно определить, как давно данный узел получал обновления.

5.6 Реконсилятор

Реконсилятор отвечает за поддержание корректного размещения целевых групп по узлам плоскости данных. Он реализован как отдельный процесс, независимый от API-сервера. Такое разделение принципиально. Если реконсилятор перезапускается или временно недоступен, API-сервер продолжает принимать запросы от клиентов и доставлять конфигурацию агентам. Новые изменения накапливаются в `etcd` и будут обработаны реконсилятором после его восстановления.

Поскольку реконсилятор модифицирует размещения в `etcd`, одновременная работа нескольких его экземпляров привела бы к конкурирующим изменениям. Для исключения этого используется выбор лидера через аренду `etcd`. Только тот экземпляр, которому удалось захватить аренду, выполняет реконсиляцию. Остальные ожидают в режиме горячего резерва и перехватывают аренду при истечении срока у лидера. Задержка переключения определяется длиной аренды и составляет несколько секунд, что вполне приемлемо для фонового процесса, не лежащего на критическом пути трафика.

Помимо управления экземплярами, реконсиляция запускается по событиям. Реконсилятор подписывается на изменения в пространстве ключей `etcd` и реагирует на три класса событий, а именно появление или изменение целевой группы, изменение статуса узла плоскости данных, а также периодический таймер для принудительной полной проверки. Благодаря событийной модели реконсилятор не выполняет периодических опросов состояния. Он пробуждается ровно тогда, когда это необходимо.

События об отказе узла обрабатываются иначе. Если узел прекратил продлевать аренду etcd, его ключ статуса удаляется и реконсилятор получает соответствующее событие. Однако немедленная реакция нежелательна. Кратковременный сбой сети или перезапуск агента не должны приводить к масштабному перераспределению целевых групп. Поэтому событие о смерти узла обрабатывается с настраиваемой задержкой `nodeDeathEventDelay`. Если за это время узел восстановился и снова зарегистрировался в etcd, событие отбрасывается без каких-либо изменений в размещении. Такой механизм существенно снижает количество лишних перемещений при кратковременных нестабильностях.

5.7 Алгоритм размещения целевых групп

Алгоритм размещения решает задачу распределения целевых групп по живым узлам плоскости данных с соблюдением фактора репликации. Каждая целевая группа должна быть размещена ровно на заданном числе узлов, а нагрузка между узлами должна быть примерно одинаковой. При этом необходимо минимизировать число перемещений. Каждое перемещение целевой группы на новый узел влечёт инкремент версии размещения и последующую реконсиляцию на агенте, что создаёт дополнительную нагрузку на систему.

Алгоритм работает в два шага. На первом шаге обрабатываются случаи недореплицированности. Целевые группы, у которых число живых размещений меньше фактора репликации, например из-за отказа узла, распределяются на случайно выбранные живые узлы до восстановления требуемого числа копий. На этом шаге применяется жадная стратегия отбора. Задача состоит в том, чтобы как можно быстрее восстановить отказоустойчивость, а не добиться идеального баланса нагрузки.

Второй шаг выравнивает нагрузку между узлами. Вычисляется целевое число целевых групп на узел, после чего применяется алгоритм двух указателей. Узлы сортируются по текущей нагрузке, и целевые группы перемещаются от наиболее нагруженных к наименее нагруженным до достижения баланса. При равной нагрузке порядок узлов определяется детерминированно по их идентификатору, что обеспечивает воспроизводимость результата при одинаковом входном состоянии. Перемещение выполняется только в том случае, если целевая группа ещё не присутствует на узле-получателе, что исключает возникновение дубликатов.

Минимизация перемещений нужна не только из соображений нагрузки. Любое перемещение целевой группы между узлами означает, что в течение интервала между удалением её с одного узла и применением на другом группа фактически размещена на меньшем числе узлов, чем требует фактор репликации. В этот короткий промежуток устойчивость к отказу одного из оставшихся узлов снижена. Кроме того, перенос группы между узлами плоскости данных перенаправляет часть клиентского трафика на новый узел, что для активных TCP-соединений увеличивает шанс их обрыва, так как на новом узле не будет актуальной таблицы соединений (для длительных соединений узел, выбранный по 5-tuple хешу, может не совпасть с тем, что зафиксирован в таблице соединений прежнего узла). По этим причинам алгоритм не стремится достичь идеального распределения любой ценой. Если текущее размещение уже достаточно близко к целевому, перемещения не выполняются, и реконсиллятор завершает цикл без изменений в etcd и без нагрузки на агентов.

5.8 Доставка конфигурации агентам

Механизм доставки конфигурации агентам построен на сочетании watch-подписок на etcd и long-poll соединений от агентов. Когда в etcd появляется изменение, затрагивающее конкретный узел плоскости данных, API-сервер получает уведомление через watch, обновляет внутренний кеш и пробуждает ожидающее соединение агента.

С точки зрения агента взаимодействие выглядит следующим образом. Агент отправляет gRPC-запрос (StreamDataPlaneAssignments), в котором указывает свою текущую версию размещения и версии всех известных ему целевых групп. API-сервер сравнивает эти версии с актуальным состоянием из кеша. Если изменений нет, запрос удерживается до появления новых данных или истечения таймаута. При наличии изменений сервер немедленно возвращает инкрементальное обновление с добавленными и удалёнными целевыми группами, изменениями спецификаций и списков эндпоинтов начиная с версий, известных агенту. Получив ответ, агент сразу устанавливает новое соединение, обеспечивая непрерывную доставку без периодического опроса.

Поскольку агент всегда сообщает, что именно ему уже известно, сервер формирует точный минимальный ответ. Это особенно заметно при переподключении после перезапуска агента. Вместо передачи полного состояния всех целевых групп сервер возвращает только то, что изменилось с момента последнего известного агенту состояния.

6 Система проверок состояния

Плоскость управления и агенты отвечают за то, чтобы конфигурация балансировщика соответствовала желаемому состоянию, однако сам по себе этот механизм ничего не знает о том, работоспособны ли серверы, к которым направляется трафик. Для решения этой задачи в архитектуре системы выделен отдельный компонент, далее называемый системой проверок состояния. Её назначение состоит в активном опросе серверов обработки трафика и своевременном информировании агентов об изменениях их доступности. В настоящем разделе описаны архитектура этой системы, алгоритм распределения проверок между узлами и механизм доставки результатов.

6.1 Существующие подходы и их ограничения

Задача мониторинга работоспособности серверов за балансировщиком не нова, и прежде чем приступить к проектированию собственного решения, были рассмотрены доступные подходы.

Пассивные проверки, применяемые в Envoy [19] под названием outlier detection, анализируют реальный трафик, и если доля ошибок у конкретного сервера превышает порог, он временно исключается из балансировки. Такой подход не требует отдельной инфраструктуры и хорошо работает в L7-окружениях. В архитектуре с DSR он принципиально неприменим, поскольку ответный трафик минует балансировщик и тот физически не видит ответов.

Помимо пассивных, активные проверки в сервисных сетках, например механизм health checking в Envoy, опрашивают серверы самостоятельно и не зависят от трафика. Они вполне применимы, однако несут накладные расходы полного HTTP-стека, избыточные для простой TCP-достижимости. Встроенные механизмы сервисных сеток к тому же ориентированы на

собственную экосистему и сложно интегрируются с произвольной инфраструктурой.

Особняком стоит подход Consul [20], реализующий децентрализованную проверку состояния с координацией через gossip-протокол. Подход с gossip-координацией был признан перспективным, однако Consul как готовое решение требует присутствия агента на каждом проверяемом узле, что не всегда возможно в облачной среде. Кроме того, он вносит внешнюю зависимость со своей моделью данных и операционными требованиями.

Исходя из вышесказанного, готового решения, удовлетворяющего всем требованиям, обнаружено не было.

6.2 Архитектура системы проверок

На рисунке 3 схематично представлена архитектура системы проверок состояния и взаимодействие её компонентов.

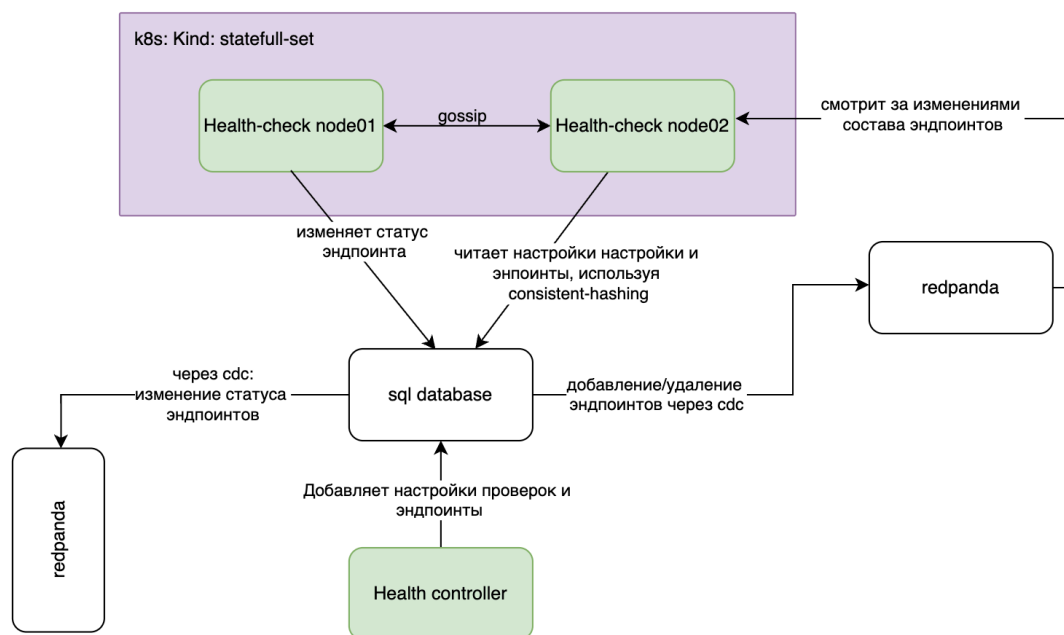


Рисунок 3 — Архитектура системы проверок состояния: узлы проверок, SQL-база данных, Redpanda и health-controller

Система проверок развёртывается как кластер равноправных узлов без выделенного лидера. Каждый узел самостоятельно выполняет проверки серверов, закреплённых за ним алгоритмом распределения. Выход из строя любого узла не нарушает работу кластера, поскольку его проверки перераспределяются между оставшимися участниками.

Узлы системы развёртываются как StatefulSet в Kubernetes. Выбор StatefulSet обусловлен требованием стабильных сетевых имён, поскольку gossip-протокол использует заранее известные адреса для начального подключения к кластеру, а предсказуемые DNS-имена упрощают конфигурацию начального списка узлов. Каждый узел получает уникальный идентификатор из имени пода.

Для координации членства в кластере был выбран протокол SWIM [10], реализованный библиотекой hashicorp/memberlist [21]. Выбор именно SWIM, а не механизма Kubernetes API или etcd обусловлен тремя соображениями. Во-первых, скорость. SWIM обнаруживает сбои через прямые опросы между узлами, тогда как Kubernetes API добавляет несколько промежуточных хопов между фактом сбоя и получением события. Во-вторых, независимость. При опросе через Kubernetes API доступность обнаружения сбоев определяется доступностью API-сервера. SWIM же работает до тех пор, пока узлы HC-кластера могут взаимодействовать между собой, и не завязан ни на etcd, ни на Kubernetes. В-третьих, переносимость. Использование SWIM позволяет развернуть систему проверок как вне Kubernetes, так и внутри него без изменений в протоколе координации. Описание самого механизма обнаружения сбоев приводится в оригинальной статье [10], а интервалы опроса и таймауты настраиваются через переменные окружения.

6.3 Распределение проверок между узлами

Центральная задача алгоритма распределения состоит в том, чтобы равномерно закрепить серверы обработки трафика за узлами проверок и корректно перераспределить их при изменении состава кластера. Простейшим решением было бы напрямую отобразить сервер на узел проверок одним консистентным хешированием. Однако число проверяемых целей на порядки превышает число узлов проверок, и при каждом изменении состава кластера хеш пришлось бы заново вычислять для каждой цели в системе. Стоимость ребаланса оказалась бы пропорциональна общему числу целей, а не числу изменившихся узлов. По этой причине от одноуровневой схемы пришлось отказаться.

Вместо этого реализована двухуровневая схема, аналогичная по устройству тому, как Cassandra [22] распределяет данные между узлами. Cassandra помещает физический узел на несколько позиций кольца посредством виртуальных узлов, и данные сначала отображаются на позицию кольца, а затем на ближайший виртуальный узел. При выходе физического узла из строя перераспределяются только его виртуальные узлы, а не весь массив данных. В разработанной системе применяется та же логика. На первом уровне каждая проверяемая цель отображается на виртуальный шард, на втором уровне виртуальный шард закрепляется за несколькими узлами проверок в соответствии с настраиваемым фактором репликации.

Закрепление цели за виртуальным шардом вычисляется один раз и может сохраняться в базе данных вместе с самой целью. При изменении состава кластера это закрепление остаётся прежним, заново вычисляется только распределение виртуальных шардов между узлами проверок. Узел, принимающий новые шарды, выбирает соответствующие цели из базы данных одним запросом по списку шардов, не обращаясь к хеш-функции

для каждой цели в отдельности. Объём ребаланса определяется числом перераспределяемых шардов, а не общим числом проверяемых целей.

Число виртуальных шардов фиксируется при запуске приложения и значительно превышает число узлов проверок. Поэтому при изменении состава кластера перераспределению подлежат только шарды, принадлежавшие изменившемуся узлу, что соответствует поведению Cassandra при добавлении или удалении физического узла. Репликация проверок означает, что каждая цель опрашивается несколькими узлами одновременно. Если один из них теряет связность с конкретным сервером, остальные реплики продолжают проверки и фиксируют корректный статус.

Для реализации консистентного хеширования поддерживаются два алгоритма. Классический вариант строит хеш-кольцо с настраиваемым числом виртуальных узлов на каждый физический. Альтернативой служит алгоритм `dxhash`, обеспечивающий равномерное распределение без дополнительной настройки числа виртуальных узлов, поскольку равномерность достигается за счёт конструкции самого алгоритма. На практике `dxhash` проще в конфигурации, поскольку исключает необходимость подбора параметров для достижения баланса.

6.4 Реакция на события членства

При присоединении нового узла в кластер узлов проверок, часть виртуальных шардов перераспределяется на новый узел, снижая нагрузку на остальных участников. Этот узел принимает шарды, загружает соответствующие цели из базы данных и начинает проверки.

При переходе узла в состояние подозреваемого, ещё до окончательного объявления о выбывании или смерти, соседи заблаговременно принимают его виртуальные шарды. Это упреждающее перераспределение сокращает разрыв между фактическим сбоем узла и

полным перехватом его проверок, что напрямую влияет на скорость реакции системы на изменение доступности серверов.

При объявлении узла выбывшим или умершим происходит окончательное переназначение шардов, и опросы возобновляются уже новыми владельцами.

6.5 Хранение и доставка результатов

Хранилище для результатов проверок должно удовлетворять одновременно нескольким требованиям. Во-первых, оно принимает поток обновлений, частота которого пропорциональна произведению числа эндпоинтов на обратную величину интервала проверок и на боевых нагрузках на порядки превышает частоту изменений собственно конфигурации балансировщика. Во-вторых, узел проверок состояния при перебалансировке должен одной операцией выбрать все цели, относящиеся к закреплённым за ним виртуальным шардам, а не обходить хранилище поэлементно. В-третьих, поток изменений нужно надёжно довести до агентов, не дублируя его через плоскость управления и не пропуская сообщений.

Этим требованиям удовлетворяет связка PostgreSQL [23] и Debezium [24]. PostgreSQL даёт реляционную модель с индексом по идентификатору виртуального шарда, что позволяет узлу проверок при изменении состава кластера выбрать все свои цели одним диапазонным запросом. Debezium считывает журнал упреждающей записи (WAL) PostgreSQL и публикует каждое изменение строки в Redpanda [25], гарантируя at-least-once-доставку без необходимости реализовывать обнаружение пропусков на стороне приложения. Альтернативный вариант с использованием etcd рассматривался и был отвергнут. Пространство ключей etcd отведено под конфигурацию с относительно редкими изменениями, а не под поток

мониторинга, и хранилище не предоставляет ни эффективного выбора по диапазону произвольного атрибута, ни WAL-based CDC. Чтение etcd напрямую агентами антипаттернично, поскольку нагружает кластер координации трафиком, не связанным с конфигурацией; проксирование же тех же данных через плоскость управления потребовало бы её участия в обработке потока статусов, что нагрузило бы её без какой-либо необходимости и противоречит общему принципу разделения потоков, описанному в разделе 5.3.

Поток статусов идёт к агентам напрямую, в обход плоскости управления. Это сделано намеренно, поскольку статусы здоровья серверов меняются с высокой частотой, и направление этого потока через плоскость управления нагрузило бы её без какой-либо необходимости. Помимо разгрузки, разделение потоков повышает отказоустойчивость системы, поскольку агенты получают обновления статусов даже при временной недоступности плоскости управления.

В качестве брокера выбран Redpanda по нескольким причинам. Redpanda реализует API, совместимый с Apache Kafka [26], что позволяет использовать стандартные клиентские библиотеки. В отличие от Apache Kafka, Redpanda написан на C++ и не требует ни JVM, ни ZooKeeper, координация встроена в сам брокер, что сокращает число внешних зависимостей. Каждый агент потребляет сообщения с уникальным идентификатором группы, соответствующим идентификатору его узла, и таким образом получает полную копию потока статусов независимо от остальных. Подробнее взаимодействие агента с этим потоком описано в разделе 4.7.

По мере роста числа серверов PostgreSQL может стать узким местом по объёму записи. В этом случае естественным путём миграции является переход на YugabyteDB [27], распределённую SQL-базу с PostgreSQL-

совместимым интерфейсом. Код приложения при этом не потребует изменений, поскольку использует стандартный драйвер. В качестве ключа шардирования для таблицы статусов целесообразно использовать хеш от пары (IP-адрес, порт), что обеспечит равномерное распределение записей. Debezium поддерживает PostgreSQL-совместимые базы, поэтому цепочка CDC также останется без изменений. В текущем масштабе системы одиночного экземпляра PostgreSQL достаточно, поэтому данный вариант рассматривается как путь эволюции.

6.6 Управление списком проверяемых целей

Система проверок получает информацию о том, какие серверы опрашивать, от плоскости управления. При добавлении сервера в целевую группу плоскость управления уведомляет кластер проверок, который включает его в список активных целей и присваивает ему виртуальный шард. При удалении сервера или целевой группы проверки прекращаются.

7 Тестирование

В данном разделе описано тестирование разработанной системы. Оно проводилось на трёх уровнях. Функциональное тестирование проверяло корректность сквозных сценариев на штатном стенде. Нагрузочное проверяло пропускную способность отдельных компонентов на изолированных бенчмарках. Хаос-тестирование проверяло поведение системы при одновременных отказах компонентов и деградации инфраструктуры. Именно для этого система и проектировалась.

7.1 Функциональное тестирование

Функциональное тестирование проверяло путь конфигурации от gRPC API плоскости управления до записи в VPP на узлах плоскости данных. Сценарии включали создание целевой группы, добавление и удаление эндпоинтов, изменение спецификации, удаление группы целиком и реакцию на изменение статуса здоровья.

Также была проверена логика реконсиляции. После серии операций добавления, удаления и обновления конфигурация VPP на агентах плоскости данных соответствовала желаемому состоянию из etcd. Нездоровые эндпоинты исключались из балансировки в пределах одного цикла реконсиляции после получения соответствующего статуса из Redpanda.

7.2 Нагрузочное тестирование

Производительность реконсильтора плоскости управления проверялась микробенчмарками, измерявшими время генерации размещения в отрыве от обращений к etcd. Полная генерация размещения с нуля для 10 000 целевых групп по 500 узлам плоскости данных занимала около 8 миллисекунд, для 100 000 групп по 1000 узлам не превышала 110

миллисекунд, а повторная реконсильция после отказа одного узла в сбалансированном кластере из 500 узлов укладывалась примерно в 3 миллисекунды, поскольку перестраивается лишь часть размещения. Результаты измерений для четырёх масштабов кластера приведены на рисунке 4. Видно, что время растёт примерно линейно с числом целевых групп, а около половины его приходится на первый шаг алгоритма, расчёт недореплицированных групп. Это подтверждает применимость двухшагового алгоритма из раздела 5.7 на масштабах реального облачного провайдера.

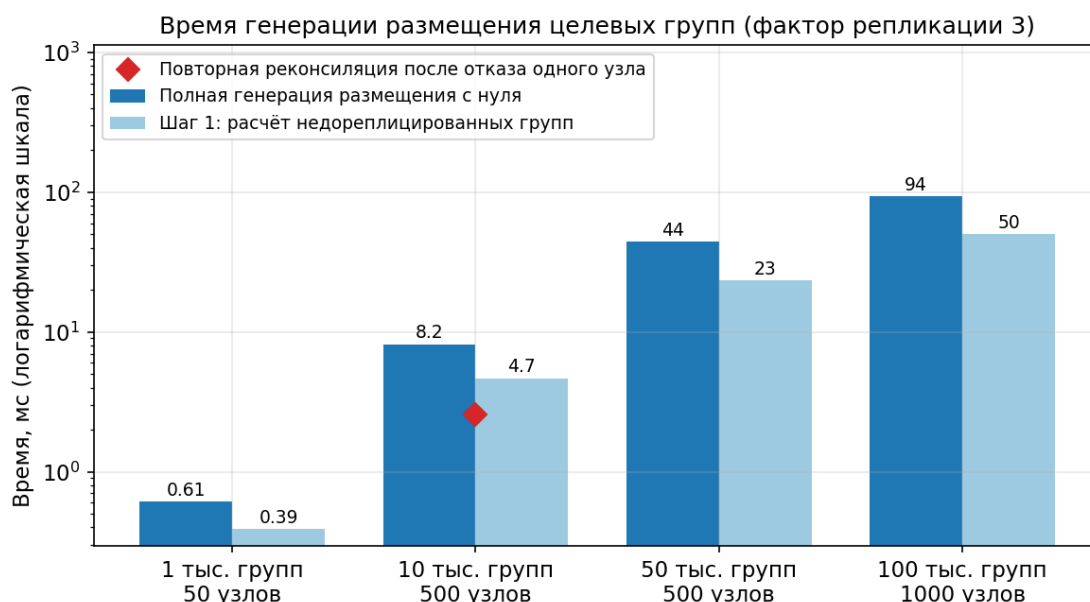


Рисунок 4 — Время генерации размещения целевых групп по масштабам кластера

Планировщик системы проверок состояния был проверен отдельным микробенчмарком, изолирующим логику планирования от выполнения самих проверок. На одном узле планировщик выполнял от 129 тысяч до 631 тысячи циклов планирования в секунду в зависимости от числа обслуживаемых целей от 1 миллиона до 1 тысячи, а время одного цикла оставалось на уровне единиц микросекунд. При 100 000 целей оно

составляло 4,6 микросекунды по p50 и 9,1 по p99. Зависимость пропускной способности и латентности цикла от числа целей приведена на рисунке 5. Рост латентности с числом целей остаётся логарифмическим, что соответствует устройству планировщика на основе двоичной кучи. Таким образом, сама по себе диспетчеризация проверок на несколько порядков дешевле минимального интервала проверок в одну секунду. Под реальной нагрузкой, когда каждая проверка выполняет сетевой запрос, узким местом оказывается не планировщик, а стандартный сетевой стек ядра Linux. Время уходит на системные вызовы создания сокетов и последующей обработки пакетов, а не на логику планирования. Это согласуется с публичными описаниями инструментов этого класса. Так, доклад Яндекса о собственной системе проверок состояния [7] тоже называет именно обработку пакетов в ядре ограничивающим фактором. Это наводит на мысли об обходе части сетевого стека Linux с помощью eBPF и AF_XDP [28]. Более точная локализация и снятие этого предела отнесены к направлениям дальнейшей работы.

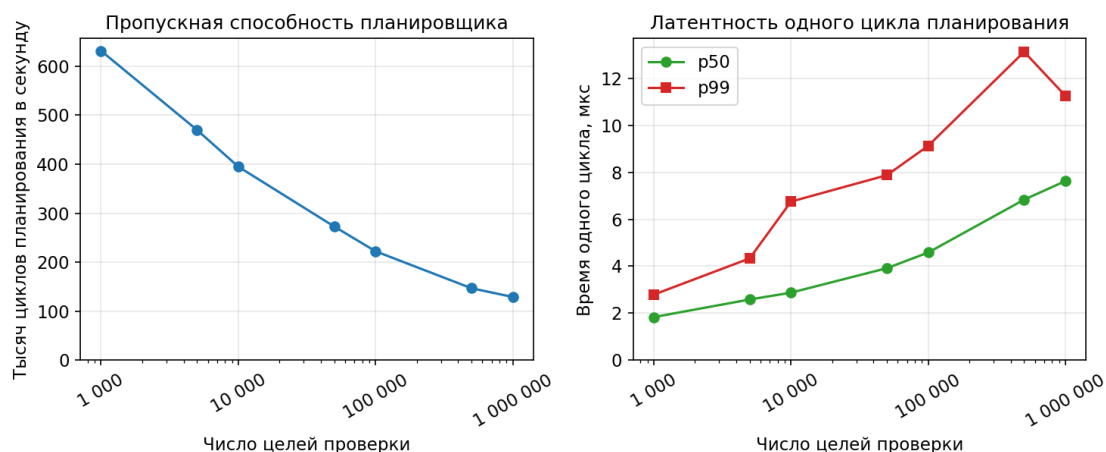


Рисунок 5 — Пропускная способность и латентность планировщика проверок состояния

7.3 Хаос-тестирование, методология

Для распределённой системы функционального и нагрузочного тестирования недостаточно. К отказоустойчивости в настоящей работе предъявляется значительное число требований, поэтому проверка поведения системы при одновременных сбоях занимает в тестировании центральное место. В реальности сбои могут происходить не по одному, иногда одновременно ломается сразу несколько компонентов, а сеть деградирует. Перебрать такие комбинации сбоев вручную в принципе возможно, однако обеспечить воспроизводимость такого перебора уже существенно сложнее, а без воспроизводимости теряется возможность сравнивать поведение разных версий системы под одним и тем же сценарием.

Прежде чем описывать сами сценарии отказов, необходимо ввести два инструмента, на которых строятся измерения. Тестовый сервер представляет собой обычное HTTP-приложение, развёрнутое за виртуальным IP-адресом балансировщика. Оно отвечает на запросы балансировщика, на проверки состояния и на пользовательский трафик. С точки зрения системы это рядовой сервер обработки, ничем не отличающийся от реальной нагрузки. В свою очередь тестовый клиент — это многопоточный генератор HTTP-нагрузки, обращающийся к виртуальному IP с настраиваемой частотой и параллелизмом. Он не привязан ни к одному конкретному серверу и пользуется ровно теми же путями балансировки, что и обычное клиентское приложение в облаке. Благодаря этому метрики, собранные на стороне тестового клиента, отражают поведение системы именно с точки зрения её конечного пользователя, что делает их пригодными для оценки SLO в условиях отказов.

Для собственно применения отказов используется Chaos Mesh [29], платформа, в которой отдельные воздействия описываются YAML-манифестами и собираются в единый сценарий эксперимента, применяемый к Kubernetes-стенду. Выбор обусловлен двумя соображениями. Во-первых, декларативное описание сценария фиксирует траекторию отказов, то есть их последовательность, длительности, правила выбора поражённых компонентов, доли поражения и паузы между фазами. Это позволяет применять один и тот же сценарий к разным версиям системы и сравнивать поведение под идентичной траекторией. Подобное сравнение использовалось в работе для оценки эффекта точечных изменений в плоскости управления, например задержки обработки события смерти узла. При той же траектории отказов меняется поведение системы в начальных фазах, тогда как в фазе тотального хаоса основной нагрузкой остаётся перераспределение виртуальных шардов в кластере проверок. Без зафиксированной траектории такое сравнение опиралось бы на доверие к усреднённым метрикам, а не на сопоставление одних и тех же интервалов времени. Во-вторых, Chaos Mesh покрывает все нужные типы воздействий, а именно убийство пода, продолжительный отказ пода, ограничение процессорных и оперативных ресурсов, сетевые задержки, потери пакетов, разделения сети и DNS-ошибки. Реализация аналогичного набора вручную потребовала бы существенно больших усилий без выигрыша в воспроизводимости.

У применённой методики есть ограничение, которое влияет на интерпретацию результатов. Chaos Mesh не предоставляет способа зафиксировать конкретный набор подов, отбираемых под воздействие при заданной доле поражения и случайном выборе процента. Поэтому фиксируется только траектория отказов по классам компонентов, но не конкретные имена подов, на которые отказ приходится. Для сравнительных

и оценочных целей этого достаточно. Усреднение по серии повторных прогонов даёт сопоставимые значения метрик. Строгий регрессионный анализ потребовал бы доработки самого инструмента.

Сами сценарии отказов организованы в четыре фазы нарастающей сложности. Первая фаза проверяет реакцию на отказ одного класса компонентов. В её состав входят сценарии убийства 75% узлов проверки состояний, отказа 50% агентов плоскости данных, убийства контейнеров тестовых серверов, выхода из строя реконсиллятора плоскости управления, задержки 200 мс до etcd и других инфраструктурных компонентов, сетевого разделения длительностью 30 секунд, а также повышенной процессорной нагрузки на агенты (порядка 60% на двух ядрах) и нагрузки на оперативную память узлов проверки состояний (порядка 256 МБ). При отказе пода агента одновременно с ним прекращает работу и плоскость данных соответствующего узла, поскольку тестовая реализация плоскости данных размещается в том же процессе. Это нужно учитывать при интерпретации клиентских ошибок в соответствующих сценариях.

Вторая фаза комбинирует два типа сбоя одновременно. В её состав входят сценарии одновременного отказа узлов проверки состояний и тестовых серверов, одновременного отказа агента плоскости данных и реконсиллятора плоскости управления, отказа узлов проверки состояний с потерями 30% в gossip-канале, задержки CDC 500 мс совместно с отказом тестовых серверов и аналогичные пары.

Третья фаза комбинирует три типа сбоя. Сюда входят сценарии одновременного отказа узлов проверки состояний, тестовых серверов и агентов, отказа агента и реконсиллятора плоскости управления вместе с инфраструктурными задержками до 300 мс, а также потерь в gossip-канале до 35% совместно с отказом узлов проверки состояний и задержками CDC.

На четвёртой фазе одновременно применяются 75% отказ узлов проверки состояний, 40% потерь в gossip-канале, 50% отказ тестовых серверов, 40% отказ агентов плоскости данных, отказ плоскости управления, инфраструктурная задержка свыше 400 мс и DNS-ошибки при обнаружении сервисов.

Между сценариями выдерживался интервал в 20 секунд для возврата затронутых подов в готовность. Между фазами выдерживалась пауза 45 секунд для полной стабилизации системы. Эти значения выбраны с большим запасом по результатам первичных измерений.

7.4 Тестовый стенд

Тестирование проводилось на Kubernetes-кластере, содержащем 2 API-сервера плоскости управления, 2 реконсилятора плоскости управления (один лидер, один в горячем резерве через etcd-аренду), 2 API-сервера системы проверок состояния, 4 узла проверки состояний (развёрнутые как StatefulSet), 4 агента плоскости данных, 16 тестовых серверов (StatefulSet с anti-affinity по узлам) и 2 тестовых клиента.

Параметры компонентов выбраны следующим образом. Фактор репликации проверок состояния равен 2, интервал проверок 3 секунды, порог обнаружения отказа 2 последовательных неуспешных проверки. Фактор репликации целевых групп равен 4. Каждый тестовый клиент генерировал постоянный поток в 25 запросов в секунду через VIP балансировщика, итого около 50 запросов в секунду суммарной нагрузки.

На рисунке 6 представлена панель мониторинга Grafana во время одного из хаос-прогонов. Верхняя полоса отражает доступность компонентов, и на ней видно, как одновременно проседают доступности узлов проверки состояний, агентов плоскости данных и тестовых серверов, а также периодически отказывают компоненты плоскости управления.

Несмотря на это, частота ошибок клиентского трафика через VIP остаётся низкой и возвращается к нулю между всплесками, а интервал покрытия проверок состояния удерживается в штатной полосе. Подробный разбор этих метрик приведён в разделах 7.5 и 7.6.

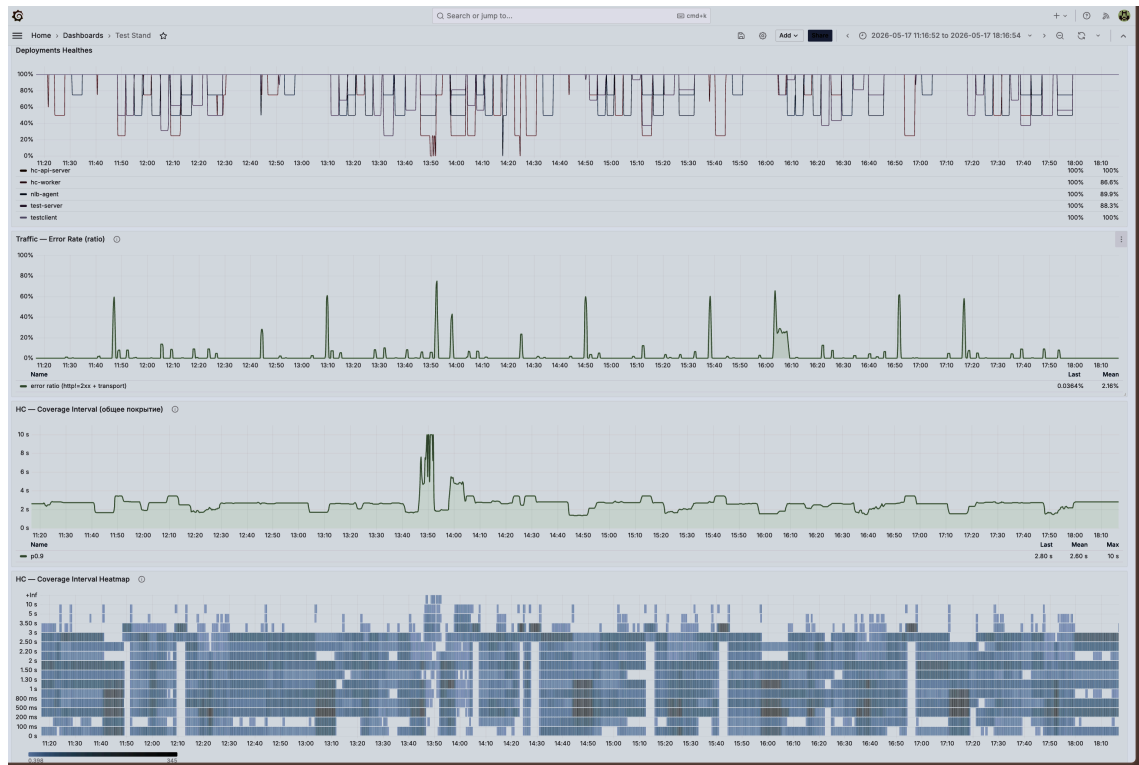


Рисунок 6 — Дашборд мониторинга Grafana во время хаос-тестирования

7.5 Метрики

Главной метрикой работоспособности системы в целом служит частота ошибок клиентского трафика `testclient_request_errors_total{kind}`, измеренная на стороне тестового клиента. Разработанная система задумывалась как балансировщик нагрузки, и именно отсутствие ошибок у клиентов отражает выполнение её прямой задачи. Внутренние сбои отдельных компонентов имеют значение лишь постольку, поскольку они приводят к ошибкам у конечного пользователя. Если ошибок нет для пользователя, то можно утверждать, что система работает корректно, какие бы перестроения ни происходили внутри. Разбивка по типу (`timeout`, `refused`,

eof, dns, other) позволяет различать классы проблемы, а парная метрика `testclient_requests_total{served_by}` показывает распределение запросов по конкретным серверам и используется для оценки равномерности балансировки.

Главной метрикой системы проверок состояния служит интервал между последовательными запросами проверки к одному и тому же серверу `testserver_hc_coverage_interval_seconds`. В штатном режиме он совпадает с настроенным периодом проверок. Рост `p90` означает, что сервер какое-то время оставался без активного мониторинга и его реальный статус не был известен балансировщику. Метрика собирается на стороне тестового сервера по факту приёма запроса и не зависит от внутренних счётчиков системы проверок. Поэтому она не теряет точность при отказах самого кластера, что важно для интерпретации результатов хаос-тестов.

7.6 Результаты

Все четыре фазы хаос-тестирования система прошла без длительных перерывов в обслуживании клиентского трафика. Основной метрикой SLO системы проверок состояния служит `Coverage Interval`. Его динамика показана на нижних панелях рисунка 6, где приведены линия `p90` и тепловая карта распределения значений. На протяжении всего прогона, включая фазу тотального хаоса, `p90` удерживается в полосе 2–3 секунды. Единичные пики до 10 секунд приходятся на моменты массового перераспределения виртуальных шардов между узлами проверки состояний и возвращаются к норме за один-два цикла проверок.

Тот же вывод подтверждается агрегацией по всем прогонам серии, представленной на рисунке 7. На нём интервал покрытия `p90` приведён отдельно для каждого сценария, дающего клиентский эффект, а также для изолированных отказов кластера проверок состояния, продолжительного

отказа подов и нагрузки на оперативную память. Во всех сценариях, включая фазу тотального хаоса, медиана по прогонам удерживается в полосе около 2–3 секунд и не приближается к бюджету обнаружения отказа в 9 секунд, который складывается из трёх пропущенных проверок при периоде в 3 секунды. При изолированной потере трёх из четырёх узлов проверки состояния интервал за один цикл выходит на ровный уровень около 3 секунд и далее не нарастает в течение всего отказа, что говорит о корректной работе перераспределения виртуальных шардов даже при значительном сокращении кластера.

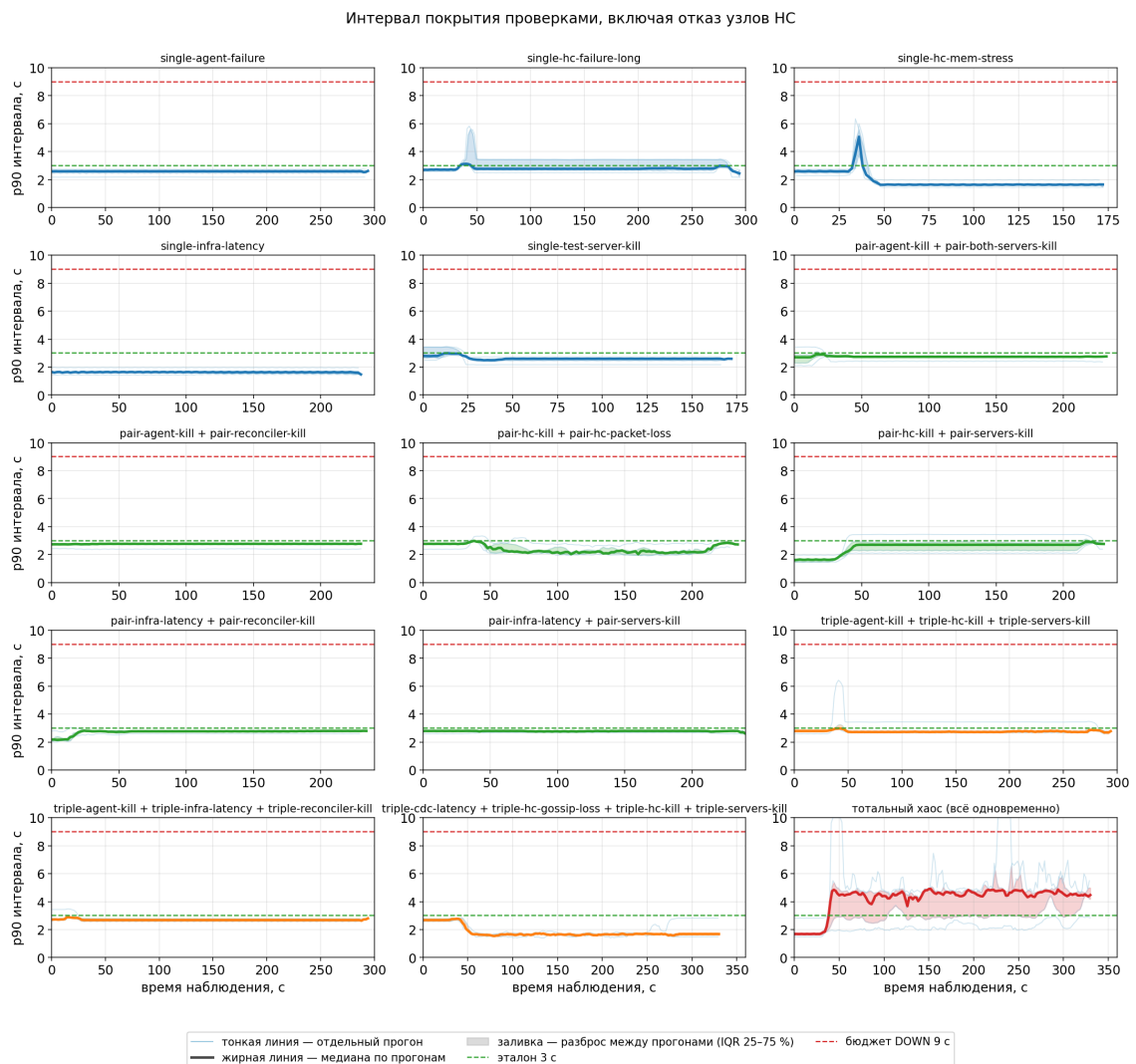


Рисунок 7 — Интервал покрытия проверками по сценариям, включая отказ узлов проверки состояния

Путь данных под хаос-нагрузкой не деградировал по времени обработки. Длительность операций VPP на агентах удерживалась на уровне около 990 мкс по р99 во всех фазах и совпадала со штатным режимом. Многослойный отказ не сказывался на скорости применения изменений в плоскости данных.

Поскольку разрабатывалась именно система балансировки нагрузки, отдельно измерялась длительность клиентского сбоя, то есть самый длинный непрерывный отрезок времени, в течение которого клиент видел ошибки. Эта величина приведена на рисунке 8 по логическим сценариям, обобщённая по серии повторных прогонов и сгруппированная по фазам хаос-тестирования. Она устойчива к редким одиночным всплескам в ходе долгого отказа и отражает реальное время восстановления балансировки с точки зрения пользователя. Значения сняты дифференцированием счётчика Prometheus с шагом выборки около 2 секунд, поэтому отрезки короче этого шага не разрешаются, а реальный перерыв в части сценариев был ещё меньше показанного. В подавляющем большинстве сценариев отрезок ошибок составляет лишь несколько секунд и возвращается к нулю за один-два шага выборки, а в наиболее тяжёлых комбинациях не превышает 12 секунд. Шесть логических сценариев не дали ни одной клиентской ошибки ни в одном прогоне. В части сценариев в отдельных прогонах отрезок ошибок не успевал закрыться до начала следующего сценария и отмечен на рисунке звёздочкой, поэтому показанная для них длительность является нижней оценкой. Сами величины при этом невелики, а наибольшие из них приходятся на фазу тотального хаоса при одновременном отказе всех классов компонентов.

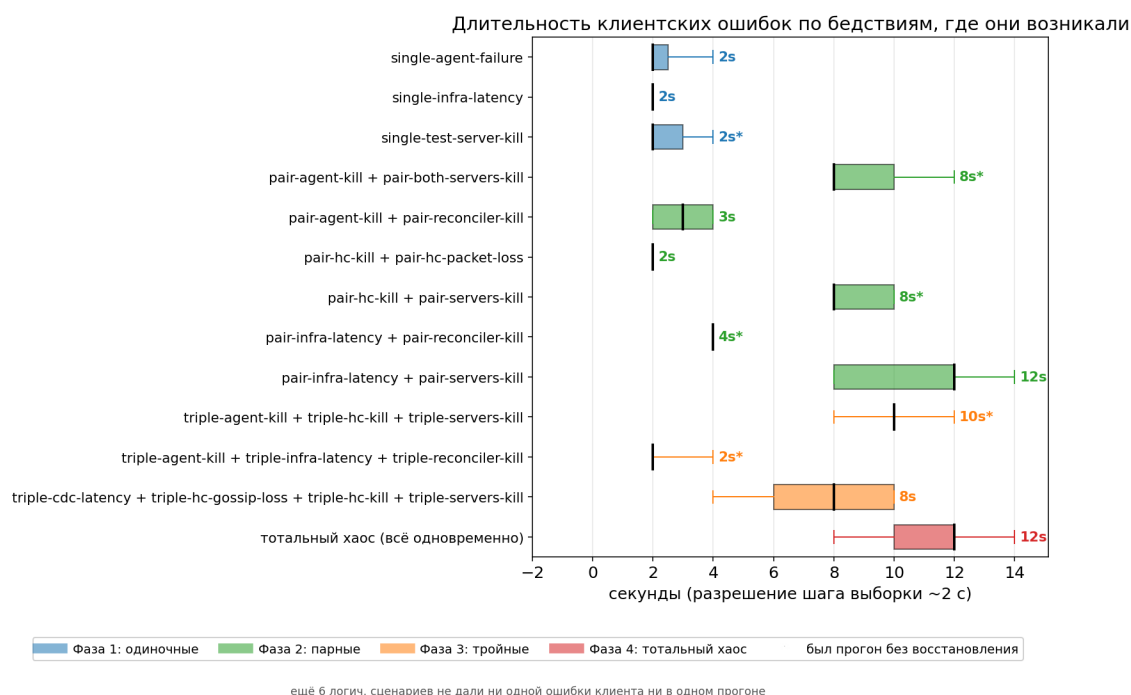


Рисунок 8 — Длительность непрерывного клиентского сбоя по сценариям и фазам

При чтении этих результатов нужно учесть один технический момент. На графиках доступности компонентов часть кратких провалов сглаживается, потому что Prometheus снимает значения с шагом, а Grafana дополнительно округляет визуализацию. Реальная просадка пакетов в моменты применения хаос-сценариев была заметно глубже того, что отражено на графиках. В фазе 4 одновременно отказывали узлы проверки состояний, агенты плоскости данных, тестовые серверы и компоненты плоскости управления. Однако клиентский трафик продолжал обрабатываться, а покрытие, измеренное на стороне тестового сервера, оставалось в пределах SLO. В этом и состоит основной результат тестирования. Предложенная архитектура поглощает многослойный отказ за счёт избыточности на каждом уровне (репликация целевых групп, репликация виртуальных шардов, дублирование экземпляров плоскости управления и системы проверок состояния) и реконсильации, восстанавливающей расхождения автоматически.

Архитектурно ожидаемые единые точки отказа подтвердились. Полная недоступность etcd ломает плоскость управления (прямое следствие выбора etcd как единственного источника истины, раздел 5.4). Цепочка PostgreSQL и Redpanda образует единую точку отказа в доставке статусов проверок (раздел 6.5). Оба случая решаются стандартными средствами, кластером etcd из 3–5 узлов и репликацией PostgreSQL. В боевой эксплуатации эти конфигурации обязательны. На тестовом стенде использовались одиночные экземпляры как достаточные для проверки логики разработанных компонентов. Их отказ выводит систему из строя предсказуемым образом, а не вскрывает дефекты архитектуры.

ЗАКЛЮЧЕНИЕ

В настоящей работе спроектирован и реализован отказоустойчивый многопользовательский L3/L4 балансировщик нагрузки на базе VPP для облачной среды. Система предоставляет виртуальные IP-адреса с автоматическим исключением неработоспособных серверов обработки трафика. Собственных единых точек отказа решение не вносит. Кластер проверок состояния координируется без центрального узла, а экземпляры плоскости управления и плоскости данных дублируются, поэтому отказ любого из них не выводит систему из строя. Немногочисленные оставшиеся зависимости сводятся к инфраструктурным хранилищам, а именно etcd и связке PostgreSQL с Redpanda, которые в промышленной конфигурации сами делаются отказоустойчивыми стандартными средствами кластеризации и репликации.

Работоспособность и отказоустойчивость решения подтверждены функциональным, нагрузочным и хаос-тестированием. Все четыре фазы хаос-тестирования, вплоть до одновременного отказа узлов проверки состояний, агентов плоскости данных, тестовых серверов и компонентов плоскости управления, система прошла без длительных перерывов в обслуживании клиентского трафика. Интервал покрытия проверками, измеренный на стороне сервера обработки, во всех фазах удерживался по p90 в полосе от двух до трёх секунд и ни разу не приблизился к бюджету обнаружения отказа в девять секунд. Наиболее длинный непрерывный отрезок клиентских ошибок в большинстве сценариев не превышал нескольких секунд, в самых тяжёлых комбинациях отказов оставался в пределах двенадцати секунд, а в шести логических сценариях клиентских ошибок не возникало вовсе. На нагрузочных измерениях двухшаговый алгоритм размещения построил полную раскладку для 10 000 целевых групп по 500 узлам примерно за 8 миллисекунд, а для 100 000 групп по 1000

узлам уложился в 110 миллисекунд, что подтверждает применимость решения на масштабах реального облачного провайдера.

В ходе работы были получены следующие результаты:

- трёхкомпонентная архитектура, спроектированная и задокументированная с формально определёнными зонами ответственности и интерфейсами взаимодействия между плоскостью управления, кластером проверок состояния и агентом плоскости данных;

- паттерн реконсиляции желаемого и фактического состояния, используемый в Kubernetes, применён системно на двух уровнях: глобальном размещении целевых групп по узлам и локальной конфигурации плоскости данных на каждом узле;

- кластер проверок состояния, реализованный без центрального координатора на основе сочетания gossip/SWIM-протокола и консистентного хеширования, что обеспечивает автоматическое обнаружение отказов и ребалансировку нагрузки без внешнего управления;

- отсутствие единой точки отказа, которое подтверждается сохранением работоспособности при одновременном отказе произвольной комбинации экземпляров плоскости управления, узлов кластера проверок состояния и агентов плоскости данных;

- методика верификации отказоустойчивости на основе воспроизводимого декларативного хаос-стенда с четырьмя фазами нарастающей сложности, охватывающего одиночные, парные, тройные и комбинированные сбои компонентов.

Результаты работы внедрены в облачной платформе MWS Cloud Platform (ООО «МВС»), что подтверждается актом о внедрении. Предложенные архитектурные решения приняты как целевые для развития продукта Virtual Private Cloud, а децентрализованная система проверок состояния запланирована к вводу в промышленную эксплуатацию в рамках

реализации сервиса interconnect. Тем самым решена и последняя из поставленных задач, интеграция разработанных компонентов в облако.

Дальнейшее развитие системы связано с несколькими направлениями. Узким местом кластера проверок состояния остаётся сетевой стек ядра Linux, и его предполагается частично обойти средствами eBPF и AF_XDP. Вычитывание каждым агентом полного потока статусов из Redpanda ограничивает масштабируемость при росте числа клиентов, поэтому намечен переход к децентрализованному хранению и адресной репликации результатов проверок. Зависимость плоскости управления от единственного кластера etcd, который при облачных масштабах сам становится ограничением производительности, предполагается снизить за счёт разнесения хранения по нескольким кластерам. Кроме того, рассматривается поддержка шардирования одной целевой группы на несколько узлов плоскости данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ITIC. Hourly Cost of Downtime 2022 [Электронный ресурс] // Information Technology Intelligence Consulting. — 2022. — Режим доступа к ресурсу: <https://itic-corp.com/tag/hourly-cost-of-downtime/> (дата обращения: 18.10.2024).
2. Eisenbud D. E. et al. Maglev: A Fast and Reliable Software Network Load Balancer [Электронный ресурс] // 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI '16). — 2016. — Режим доступа к ресурсу: <https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eisenbud> (дата обращения: 12.10.2024).
3. Katran: A high performance layer 4 load balancer [Электронный ресурс] — Режим доступа к ресурсу: <https://github.com/facebookincubator/katran> (дата обращения: 14.10.2024).
4. IPVS — IP Virtual Server [Электронный ресурс] — Режим доступа к ресурсу: <http://www.linuxvirtualserver.org/software/ipvs.html> (дата обращения: 19.10.2024).
5. Kubernetes [Электронный ресурс] — Режим доступа к ресурсу: <https://kubernetes.io/> (дата обращения: 03.11.2024).
6. Cloudflare Unimog: Unimog: Cloudflare's edge load balancer [Электронный ресурс] — 2020. — Режим доступа к ресурсу: <https://blog.cloudflare.com/unimog-cloudflares-edge-load-balancer/> (дата обращения: 21.10.2024).
7. Балансировщик нагрузки в Яндекс Облаке [Электронный ресурс] — 2023. — Режим доступа к ресурсу: <https://habr.com/ru/companies/yandex/articles/735518/> (дата обращения: 28.10.2024).
8. FD.io — Vector Packet Processing [Электронный ресурс] — Режим доступа к ресурсу:

<https://fd.io/docs/whitepapers/FDioVPPwhitepaperJuly2017.pdf> (дата обращения: 25.10.2024).

9. Yandex Cloud SLA для балансировщиков нагрузки [Электронный ресурс] — Режим доступа к ресурсу: https://yandex.ru/legal/cloud_sla_apploadbalancer/ru/ (дата обращения: 16.11.2024).

10. Das A., Gupta I., Motivala A. SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol [Электронный ресурс] // Proceedings of the International Conference on Dependable Systems and Networks. — 2002. — Режим доступа к ресурсу: <https://courses.physics.illinois.edu/cs425/fa2020/SWIM.DSN02.pdf> (дата обращения: 09.12.2024).

11. Ferguson A. D. et al. Orion: Google's Software-Defined Networking Control Plane [Электронный ресурс] // 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI '21). — 2021. — Режим доступа к ресурсу: <https://www.usenix.org/conference/nsdi21/presentation/ferguson> (дата обращения: 02.12.2024).

12. Burns B., Grant B., Oppenheimer D., Brewer E., Wilkes J. Borg, Omega, and Kubernetes [Электронный ресурс] // Communications of the ACM. — 2016. — Т. 59, № 5. — С. 50–57. — Режим доступа к ресурсу: <https://dl.acm.org/doi/10.1145/2890784> (дата обращения: 11.12.2024).

13. Intel: A VPP-Based Layer 4 Load Balancer Technology Guide [Электронный ресурс] — Режим доступа к ресурсу: https://cdrdv2-public.intel.com/793188/high_density_scalable_load_balancer_vpp_based_layer_4_technology_guide_793188v1.pdf (дата обращения: 27.10.2024).

14. DPDK: Data Plane Development Kit [Электронный ресурс] — Режим доступа к ресурсу: <https://www.dpdk.org/> (дата обращения: 26.10.2024).
15. GoVPP: Go API for VPP [Электронный ресурс] — Режим доступа к ресурсу: <https://github.com/FDio/govpp> (дата обращения: 15.01.2025).
16. etcd [Электронный ресурс] — Режим доступа к ресурсу: <https://etcd.io/> (дата обращения: 20.01.2025).
17. Burrows M. The Chubby Lock Service for Loosely-Coupled Distributed Systems [Электронный ресурс] // 7th USENIX Symposium on Operating Systems Design and Implementation. — 2006. — Режим доступа к ресурсу: <https://static.googleusercontent.com/media/research.google.com/ru//archive/chubby-osdi06.pdf> (дата обращения: 11.02.2025).
18. Ongaro D., Ousterhout J. In Search of an Understandable Consensus Algorithm (Raft) [Электронный ресурс] // USENIX Annual Technical Conference. — 2014. — Режим доступа к ресурсу: <https://raft.github.io/raft.pdf> (дата обращения: 23.12.2024).
19. Envoy Proxy [Электронный ресурс] — Режим доступа к ресурсу: <https://www.envoyproxy.io/> (дата обращения: 03.02.2025).
20. Consul [Электронный ресурс] — Режим доступа к ресурсу: <https://www.consul.io/> (дата обращения: 05.02.2025).
21. hashicorp/memberlist: Golang package for cluster membership and failure detection [Электронный ресурс] — Режим доступа к ресурсу: <https://github.com/hashicorp/memberlist> (дата обращения: 22.01.2025).
22. Lakshman A., Malik P. Cassandra: A Decentralized Structured Storage System [Текст] // ACM SIGOPS Operating Systems Review. — 2010. — Т. 44, № 2. — С. 35–40.

23. PostgreSQL [Электронный ресурс] — Режим доступа к ресурсу: <https://www.postgresql.org/> (дата обращения: 18.01.2025).
24. Debezium [Электронный ресурс] — Режим доступа к ресурсу: <https://debezium.io/> (дата обращения: 06.02.2025).
25. Redpanda [Электронный ресурс] — Режим доступа к ресурсу: <https://github.com/redpanda-data/redpanda> (дата обращения: 07.02.2025).
26. Kreps J., Narkhede N., Rao J. Kafka: a Distributed Messaging System for Log Processing [Текст] // Proceedings of the NetDB Workshop. — 2011.
27. YugabyteDB [Электронный ресурс] — Режим доступа к ресурсу: <https://www.yugabyte.com/> (дата обращения: 14.02.2025).
28. Castillon du Perron K., Lopez Pacheco D., Huet F. Understanding Delays in AF_XDP-based Applications [Электронный ресурс] // IEEE International Conference on Communications (ICC). — 2024. — Режим доступа к ресурсу: <https://arxiv.org/abs/2402.10513> (дата обращения: 16.03.2025).
29. Chaos Mesh [Электронный ресурс] — Режим доступа к ресурсу: <https://chaos-mesh.org/> (дата обращения: 02.03.2025).